



Tsinghua University

Modern Computer Architecture

Fall 2025

Lecture 05: SuperScalar Processor Model

Shuwen Deng

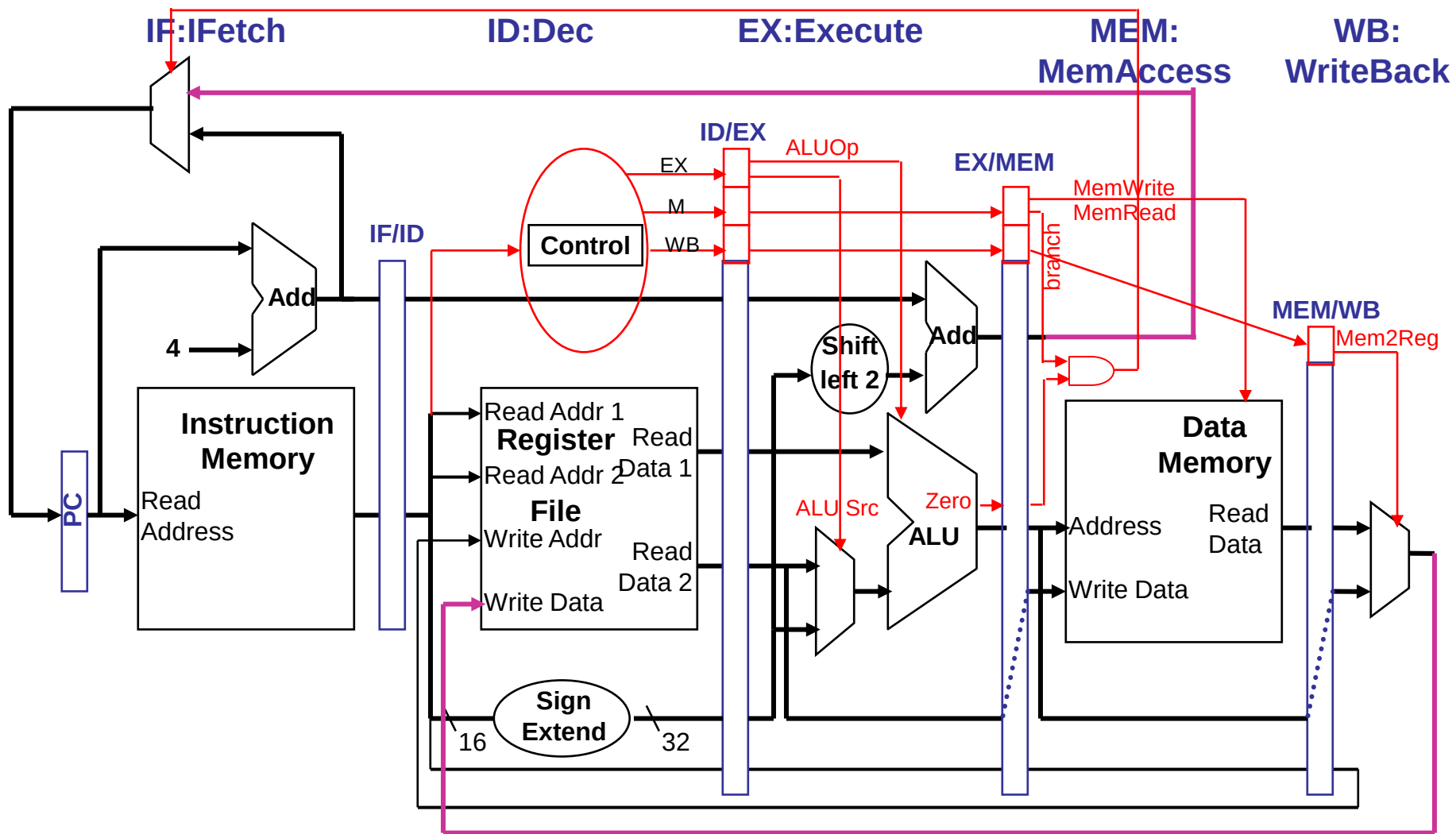
shuwend@tsinghua.edu.cn

2025-Oct.-22

Today's outline

- Review:
 - Basic pipeline
 - Issue & Completion
 - VLIW
- SuperScalar
 - From Basic Pipeline to SS
 - RUU (function, example)
 - LSQ

Review : Basic MIPS Pipeline DataPath



Review: Extracting Yet *More* Performance

- Two options:
 - Lower CCT / Higher CR :: **superpipelining**
(a brief intro last week)
 - Increase the depth of the pipeline to increase the clock rate
 - Lower CPI / Higher IPC :: **multiple-issue** :: parallel
(Main Topic of these three weeks)
 - Fetch (and execute) more than one instructions at one time
(expand every pipeline stage to accommodate multiple instructions)

$$\text{CPU Time} = \text{IC} \times \text{CPI} \times \text{CCT}$$

Instruction v.s. Machine Parallelism

- **Instruction-level parallelism (ILP)** of a program – a measure of the average number of instructions in a program that a processor *might* be able to execute at the same time
 - Mostly determined by the number of true (data) dependencies and procedural (control) dependencies in relation to the number of other instructions
- **Machine-level parallelism** of a processor – a measure of the ability of the processor to take advantage of the ILP of the program
 - Determined by the number of instructions that can be fetched and executed at the same time
- To achieve high performance, need **both** ILP and machine level parallelism

Review: Instruction Issue and Completion

- **Instruction-issue** – initiate execution
 - **Instruction lookahead** capability – ability of the processor to fetch, decode and issue instructions beyond the current point of execution to try to find more instructions to issue
- **Instruction-completion** – complete execution
 - **Processor lookahead** capability – ability of the processor to examine issued instructions beyond the current point of execution to try to find more instructions to complete

True data dependency : IOI-IOC (RbW)

Output dependency : IOI-OOC (WbW)

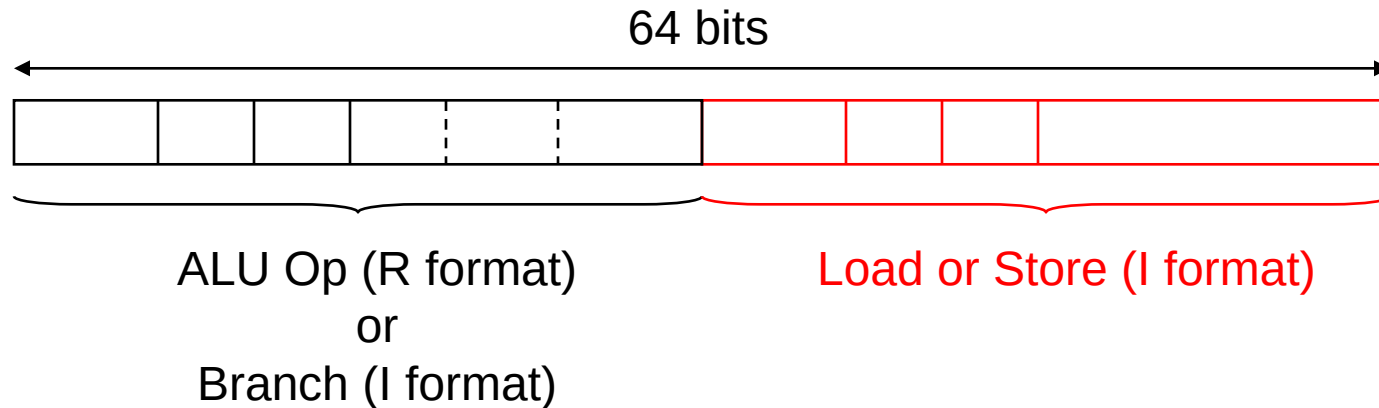
Anti dependency : OOI-OOC (WbR)

Multiple-Issue Processor Styles

- Static multiple-issue processors
(aka **Very Long Instruction Word, VLIW**)
 - Decisions on which instructions to execute simultaneously are being made **statically** (at ? time by the ?)
 - E.g. Intel Itanium and Itanium 2 for the IA-64 ISA-EPIC (Explicit Parallel Instruction Computer), and TI TMS320C6x
- Dynamic multiple-issue processors (aka **SuperScalar, SS**)
 - Decisions on which instructions to execute simultaneously are being made **dynamically** (at ? time by the ?)
 - E.g. IBM Power 2, Pentium 4, MIPS R10K, HP PA 8500

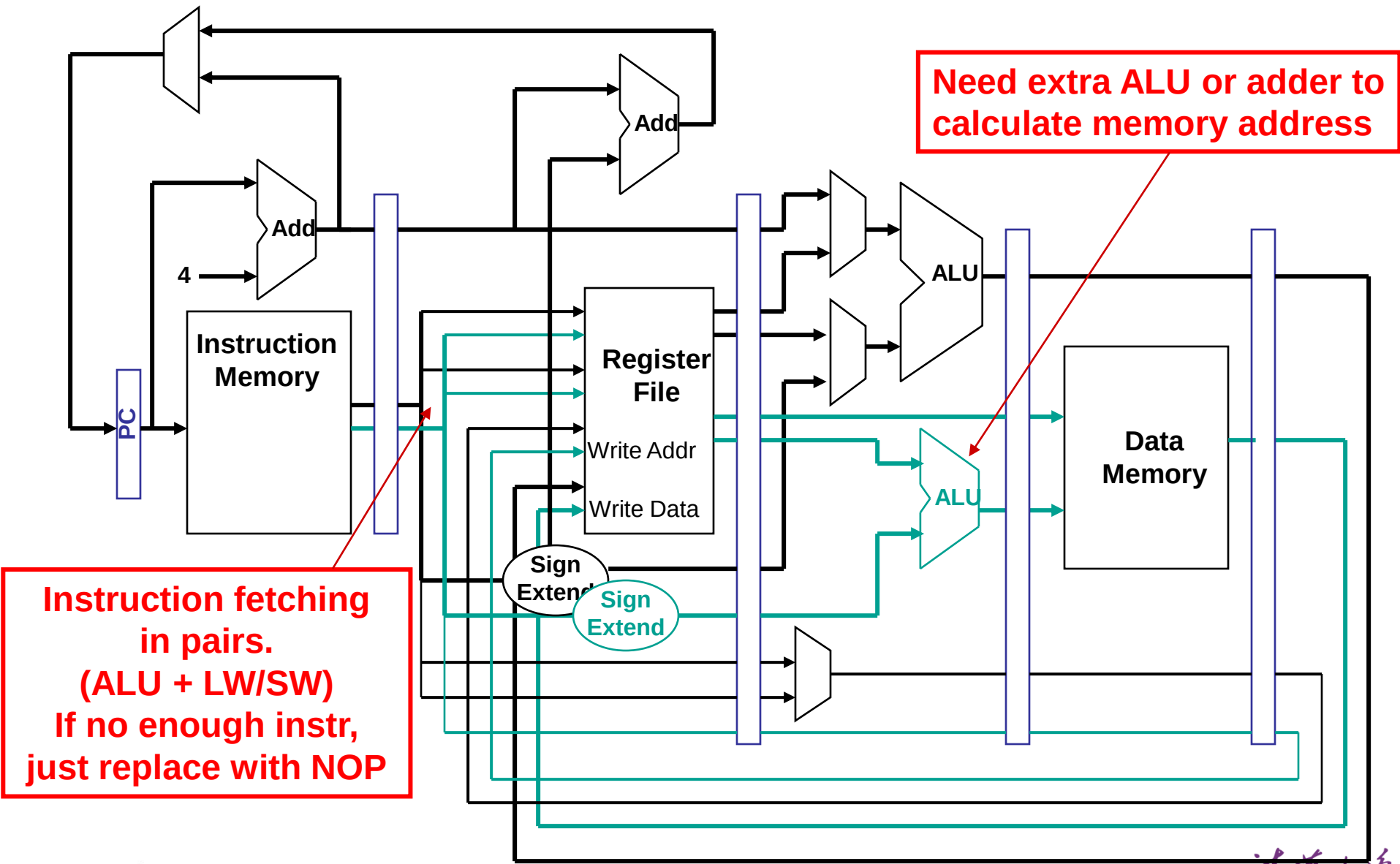
Review: Bundle Example: A VLIW MIPS

- Consider a 2-issue MIPS with a 2-instr bundle



- Instructions are always fetched, decoded, and issued in pairs
 - If one instr of the pair can not be used, it is replaced with a **noop**
- Need **4 read ports** and **2 write ports** and a separate memory address adder

Review: DataPath Example: A VILW MIPS



The Scheduled Code (Not Unrolled)

	ALU or branch	Data transfer	CC
lp:	NOOP	1. lw \$t0, 0(\$s1)	1
	4. addi \$s1, \$s1, -4	NOOP	2
	2. addu \$t0, \$t0, \$s2	NOOP	3
	5. bne \$s1, \$0, lp	3. sw \$t0, 4(\$s1)	4

- Four clock cycles to execute 5 instructions for a
 - CPI of 0.8 (versus the best case of 0.5)
 - IPC of 1.25 (versus the best case of 2.0)
 - noops don't count towards performance !!

Loop Unrolling

- Loop unrolling – multiple copies of the loop body are made and instructions from **different iterations** are scheduled together as a way to increase ILP
- Apply loop unrolling (4 times for our example) and then **schedule** the resulting code
 - Eliminate unnecessary loop overhead instructions
 - Schedule so as to avoid load use hazards
- During unrolling the compiler applies **register renaming** to eliminate all data dependencies that are not true dependencies

Predication vs. Speculation

- Got a branch? Two possible paths of execution? Not sure what to do?
 - Predication: “Do ‘em both! We’ll figure it out at the end.”
 - Speculation: “Make your best guess. We’ll figure it out at the end.”

Predication (Advanced topic)

- Predication can be used to eliminate branches by making the execution of an instruction dependent on a “predicate”, e.g.,

```
if (p) {statement 1} else {statement 2}
```

would normally compile using two branches. With predication it would compile as

```
(p) statement 1
```

```
(~p) statement 2
```

- The use of `(condition)` indicates that the instruction is committed only if `condition` is true
- Predication can be used to **speculate** as well as to **eliminate branches**

Speculation (Advanced topic)

- Speculation is used to allow execution of future instr's that (may) depend on the speculated instruction
 - Speculate on the outcome of a conditional branch (**branch prediction**)
 - Speculate that a store (for which we don't yet know the address) that precedes a load does not refer to the same address, allowing the load to be scheduled before the store (**load speculation**)
- Must have (hardware and/or software) mechanisms for
 - **Detection**: Checking to see if the guess was correct
 - **Correction**: Recovering from the effects of the instructions that were executed speculatively if the guess was incorrect
 - In a VLIW processor the compiler can insert additional instr's that check the accuracy of the speculation and can provide a fix-up routine to use when the speculation was incorrect

Compiler: Static Prediction

- Predict at compile time whether branches will be taken before execution
- Schemes
 - Predict taken
 - Backwards taken, forwards not taken (good performance for loops)
- No run-time adaptation: bad performance for data- dependent branches

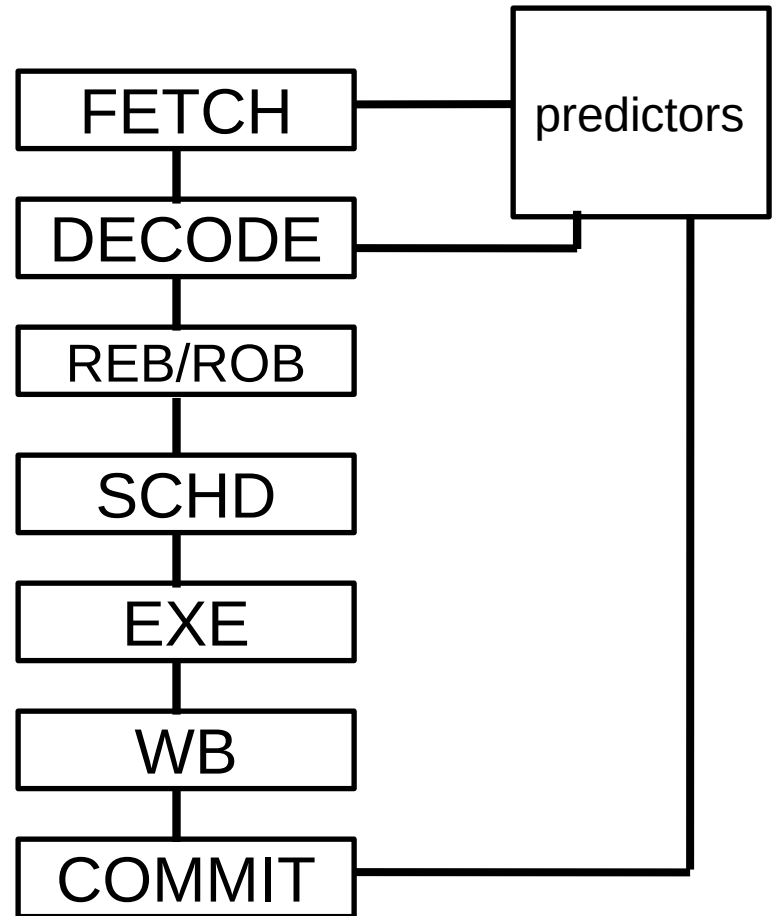
```
if (a == 0) b =3; else b=4;
```

Hardware-based Dynamic Branch Prediction

- Single level (Simple counters) – predict outcome based on past branch behavior
 - FSM (Finite State Machine)
- Global Branch Correlation – track relations between branches
 - GAs
 - Gshare
- Local Correlation – predict outcome based on past branch behavior PATTERN
 - PAs
- Hybrid predictors (combination of local and global)
- Miscellaneous
 - Return Address Stack (RAS)
 - Indirect jump prediction

Mis-prediction Detections and Feedbacks

- Detections:
 - At the end of decoding
 - Target address known at decoding, and does not match
 - Flush fetch stage
- At commit (most cases)
 - Wrong branch direction or target address does not match
 - Flush the whole pipeline
- Feedbacks:
 - Any time a mis-prediction is detected
 - At a branch's commit



1-bit “Self Correlating” Predictor

- Let's consider a simple model. Store a bit per branch:
“last time, was the branch taken or not”.
- Consider a loop of 10 iterations before exit:

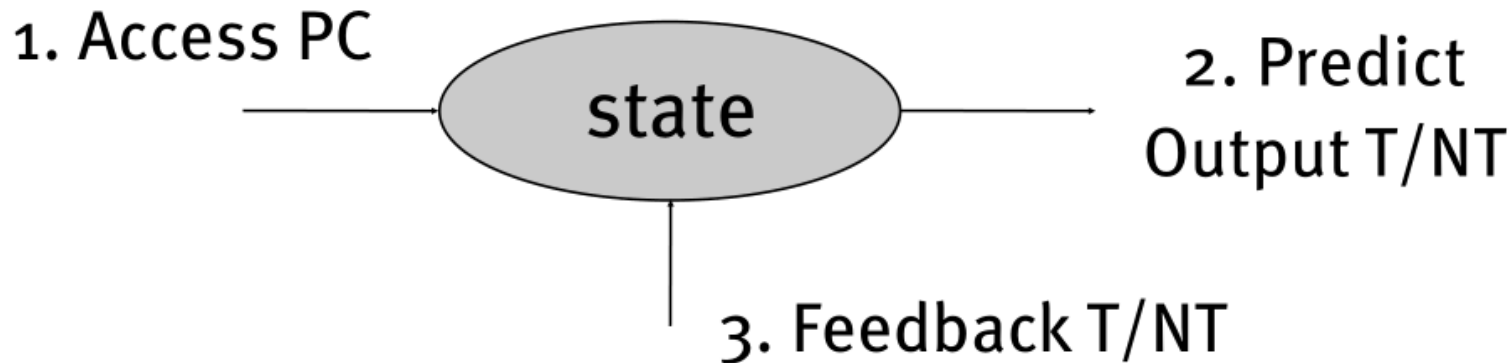
```
for (...  
    for (i=0; i<10; i++)  
        a[i] = a[i] * 2.0;
```
- What's the accuracy of this predictor?

Dynamic Branch Prediction

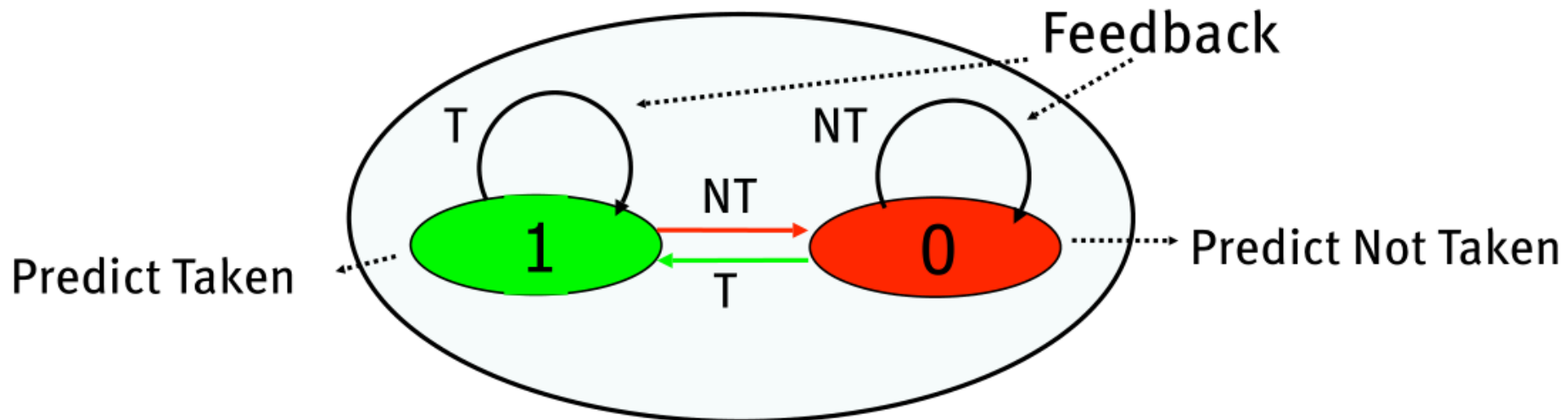
- Performance = $f(\text{accuracy, cost of misprediction})$
- Branch History Table: Lower bits of PC address index table of 1-bit values
 - Says whether or not branch taken last time
 - No address check
- Problem: in a loop, 1-bit BHT will cause two mispredictions (avg is 9 iterations before exit):
 - End of loop case, when it exits instead of looping as before
 - First time through loop on next time through code, when it predicts exit instead of looping

Predictor for a Single Branch

General Form

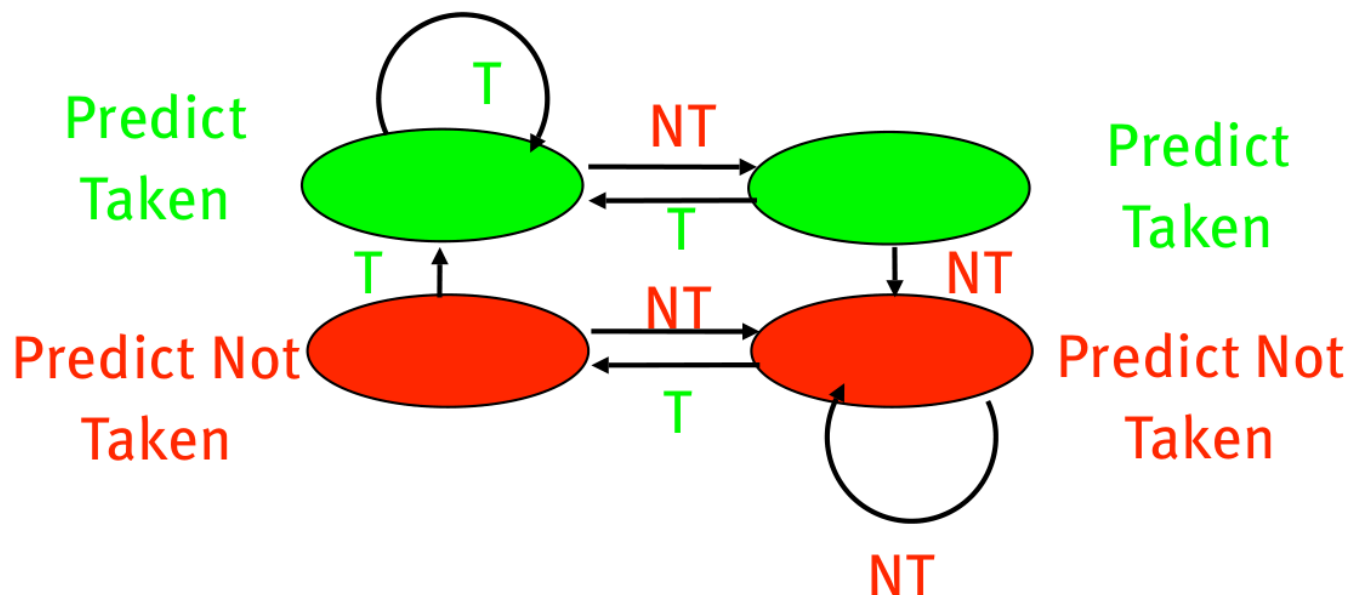


1-bit prediction



Dynamic Branch Prediction

- Solution: 2-bit scheme where change prediction only if get misprediction twice:



- Red: stop, not taken
- Green: go, taken
- Adds hysteresis to decision making process

Some Interesting Patterns

- Format: Not-taken (N) (0), Taken (T)(1)
- TTTTTTTTTT
 - 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 ... :
 - Should give perfect prediction
- NNTTNNTTNNTT
 - 0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 ... :
 - Will mispredict all of the time
- N*N[TNTN]
 - 0 0 0 0 0 0 0 0 1 0 1 0 1 0 1 0 1 0 1 0 ... :
 - Should alternate incorrectly
- N*T[TNTN]
 - 0 0 0 0 0 0 0 1 1 0 1 0 1 0 1 0 1 0 1 0 ... :
 - Should alternate incorrectly

Branch Prediction Summary

- Consider for each branch prediction
 - Hardware cost
 - Prediction accuracy
 - Warm-up time
 - Correlation
 - Interference
 - Time to generate prediction
- Application behavior determines number of branches
 - More control intensive program...more opportunity to mispredict
- What if a compiler/architecture could eliminate branches?

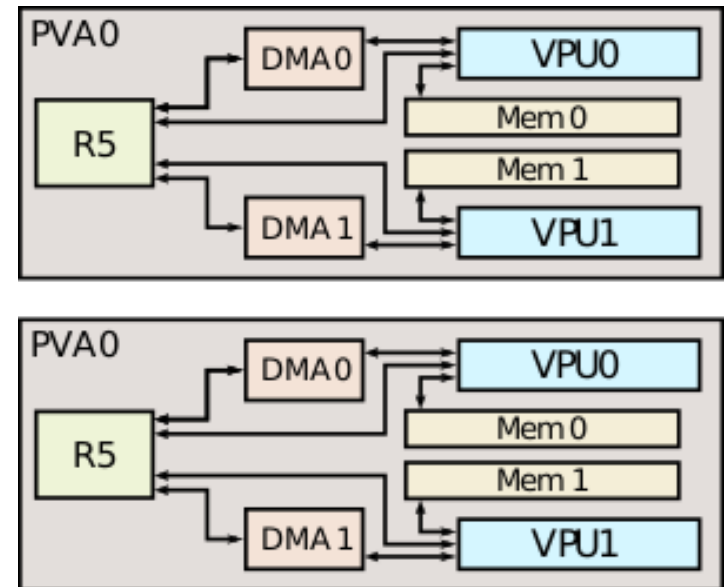
Modern VLIW Example

- Tegra Xavier – Nvidia

A 64-bit ARM high-performance system on a chip for autonomous machines designed introduced in 2018

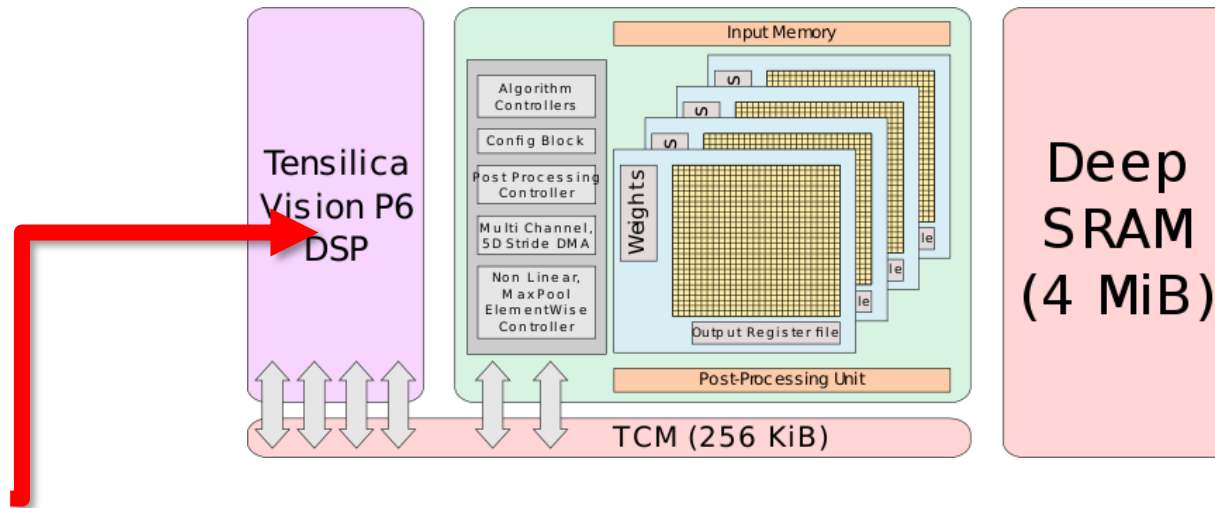
- Programmable Vision Accelerator

- To implement algorithms such as Harris corner, FFTs
- 7-slot VLIW architecture:
 - 2 scalar slots
 - 2 vector slots
 - 3 memory operations



Modern VLIW Example

- Spring Hill (SPH) – Intel
A 10 nm microarchitecture designed for inference neural processors, introduced in 2019



- Programmable vector processor
 - A customized Cadence Tensilica Vision P6 DSP
 - Added to each ICE(Inference Compute Engine) for additional flexibility
 - 5-slot VLIW 512-bit vector processor

Today's outline

- Review:
 - Basic pipeline
 - Issue & Completion
 - VLIW
- SuperScalar
 - From Basic Pipeline to SS
 - RUU (function, example)
 - LSQ

Review: Instruction Issue and Completion

- **Instruction-issue** – initiate execution
 - **Instruction lookahead** capability – ability of the processor to fetch, decode and issue instructions beyond the current point of execution to try to find more instructions to issue
- **Instruction-completion** – complete execution
 - **Processor lookahead** capability – ability of the processor to examine issued instructions beyond the current point of execution to try to find more instructions to complete

True data dependency : IOI-IOC (RaW)

Output dependency : IOI-OOC (WaW)

Anti dependency : OOI-OOC (WaR)

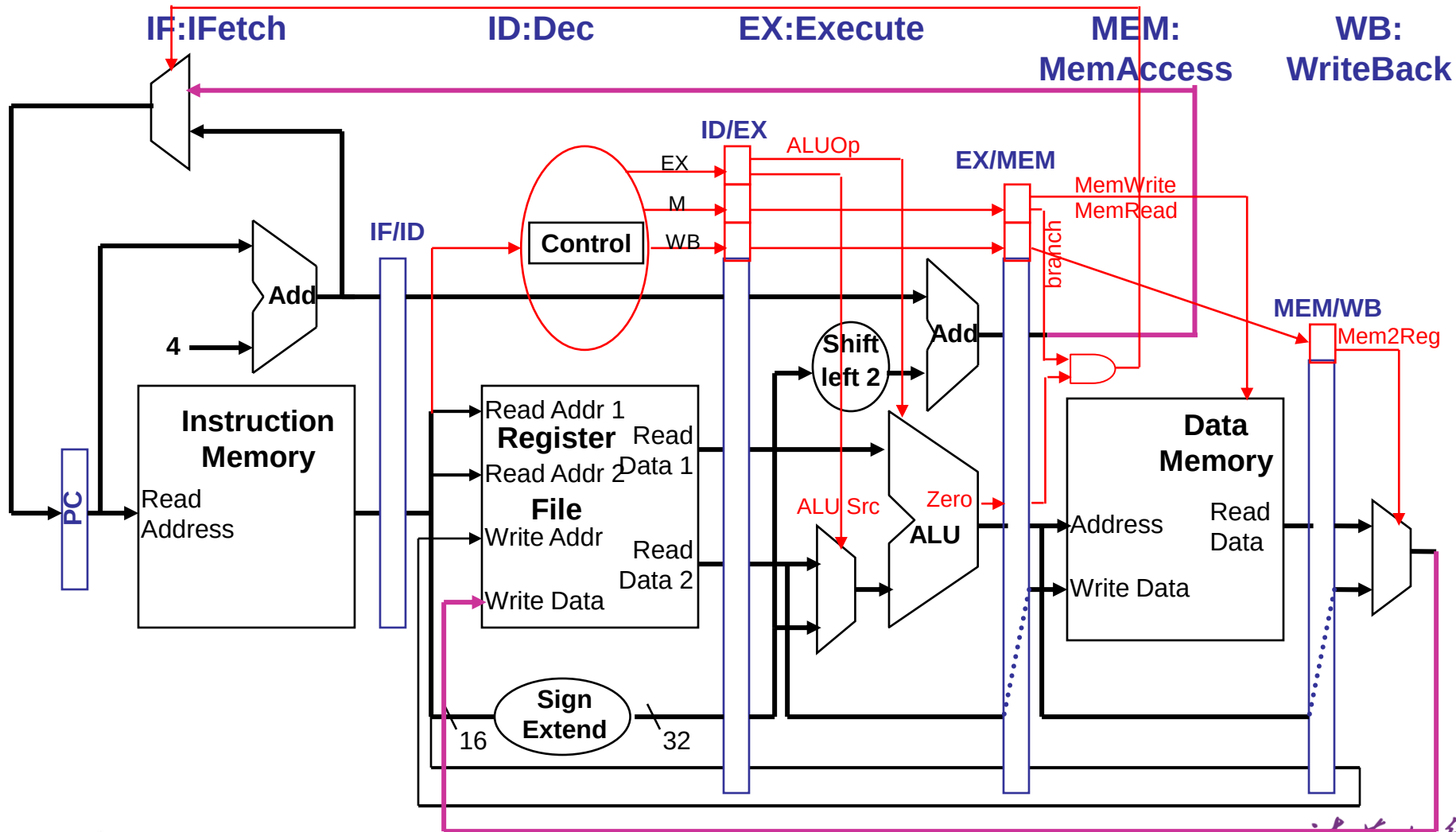
Typical Functional Unit Latencies

- ❑ **Result latency** – number of cycles taken by a functional unit (FU) to produce a result
- ❑ **Issue latency** – minimum number of cycles between the issuing of an instruction to a FU and the issuing of the next instruction to the same FU

	Issue Latency	Result Latency
Integer ALU	1	1
Integer multiply	1	2
Load (on hit)	1	1
Load (on miss)	1	40
Store	1	n/a
FltPt Add	1	2
FltPt Multiply	1	4
FltPt Divide	12	12
FltPt Convert	1	2

Review : Basic MIPS Pipeline DataPath

- How to Modify the basic datapath to implement SuperScalar?

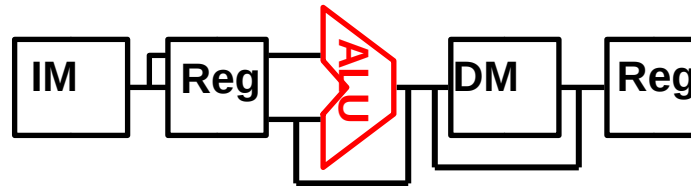


Basic Pipeline to SuperScalar

- Consider the ALU

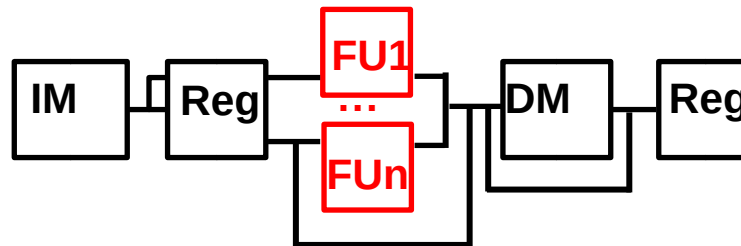
Pre: One Instr is executing at one time.

=> Use MUX to decide which function unit to use



Now: Multi Instr's are executing at one time.

=> Separate the ALU part into multi Function Unit for parallelism.



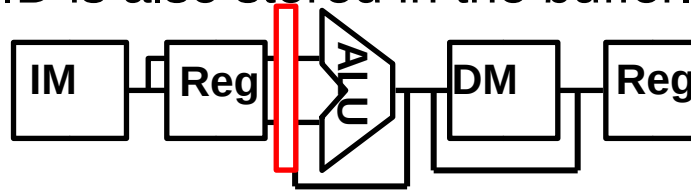
How to avoid the resource conflict of these FUs and the WriteBack Path?

Basic Pipeline to SuperScalar

- Consider the buffer between ID/EX

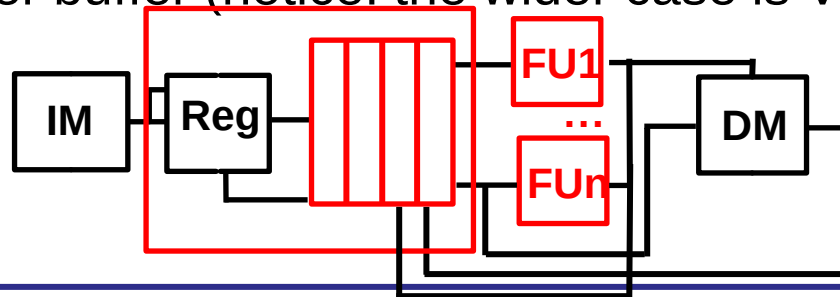
Pre: Only One Entry corresponding to One Instr can be dispatched to the buffer at one time.

=> Besides the SrcOperand and Immediate Num, CtrlBits decoded in the step of ID is also stored in the buffer.



Now: Multi Entries corresponding to Multi Instr's are dispatched to the buffer and waiting for issue in the buffer.

=> Deeper buffer (notice: the wider case is VLIW)



How to design the data structure of one Entry and the deeper buffer?
How to select the instructions to issue? Any changes on Reg? Data Dependency?

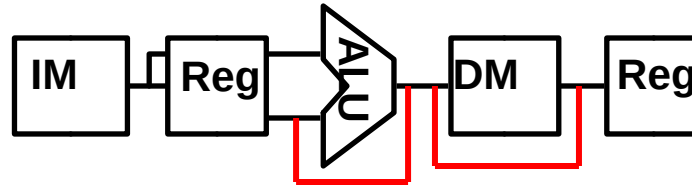
Basic Pipeline to SuperScalar

- Consider the LW/SW instr's, who will decide the load fwding?

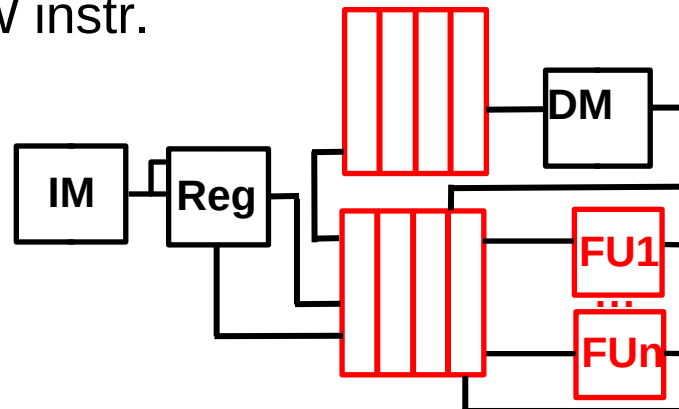
Notice: Only LW/SW reads/writes DataMem and

LW/SW just uses ALU to cal the incre address.

=> Notice two direct connection.



Now: Separate the LW/SW into 2 (micro)instr's. One for cal the incre address; one for load/store(interact with dataMemory). Introduce another queue for LW/SW instr.



Will Memory face data dependency problems?

Baseline Superscalar MIPS Processor Model

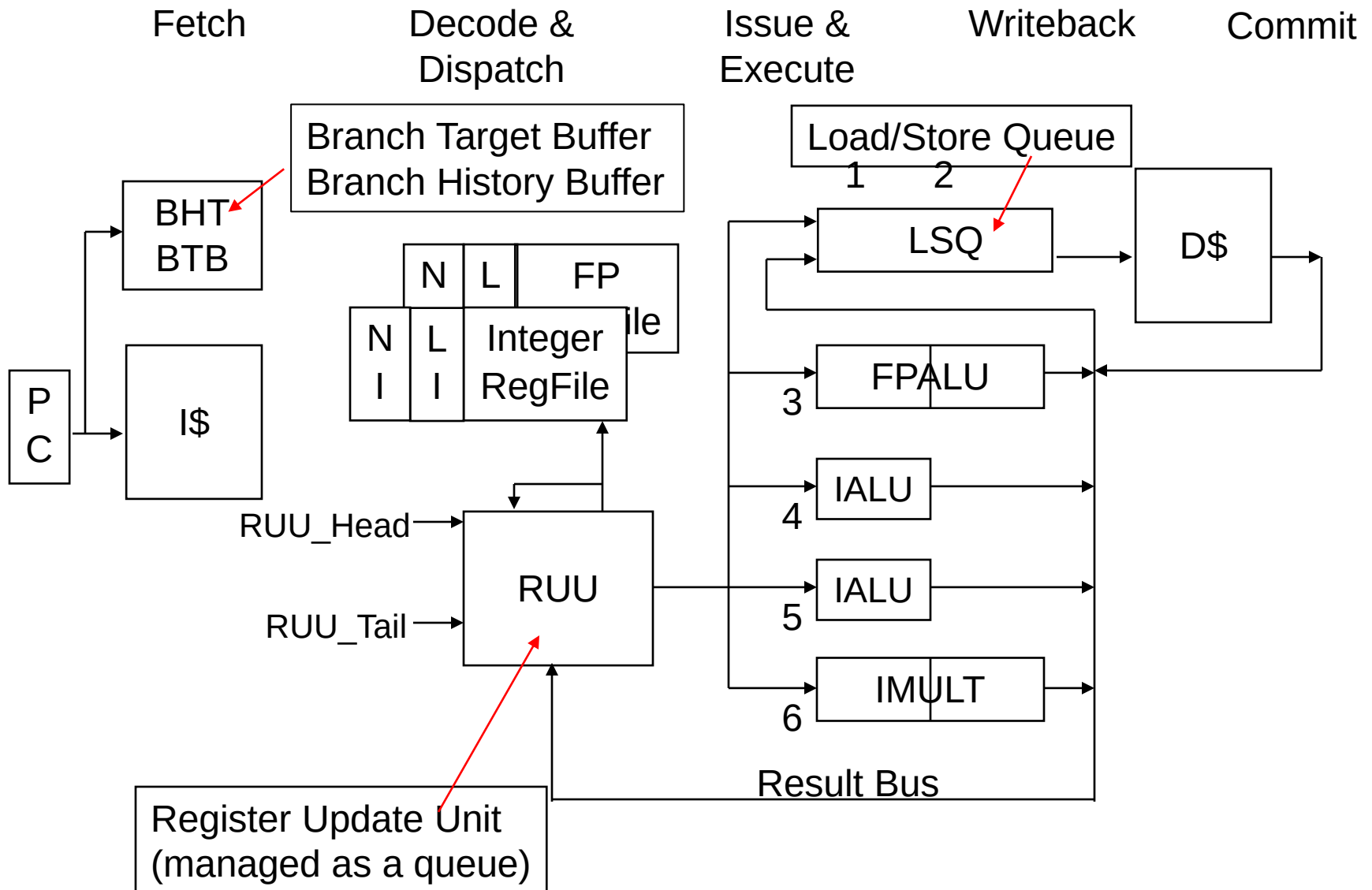


Fig. 9 from Sohi's paper

Basic Instruction Flow Overview

- Fetch (in\out of program order):
 - Fetch multiple instructions in parallel from the I\$
- Decode & Dispatch (in\out of program order):
 - In parallel, decode the instr's just fetched and schedule them for execution by dispatching them to the RUU
 - Loads and stores are dispatched as two (micro)instr's – one to compute the effective addr and one to do the memory operation
- Issue & Execute (in\out of program order):
 - As soon as the RUU has the instr's source data and the FU is free, the instr's are issued to the FU for execution
- Writeback (in\out of program order):
 - When done the FU puts its results on the Result Bus which allows the RUU and the LSQ to be updated – the instr completes
- Commit (in\out of program order):
 - When appropriate, commit the instr's result data to the state locations (i.e., update D\$ and RegFile)

Basic Instruction Flow Overview

- Fetch (~~in~~~~out~~~~of~~ program order):
 - **Fetch** multiple instructions in parallel from the I\$
- Decode & Dispatch (~~in~~~~out~~~~of~~ program order):
 - In parallel, **decode** the instr's just fetched and schedule them for execution by **dispatching** them to the RUU
 - Loads and stores are dispatched as two (micro)instr's – one to compute the effective addr and one to do the memory operation
- Issue & Execute (~~in~~~~out~~ of program order):
 - As soon as the RUU has the instr's source data and the FU is free, the instr's are **issued** to the FU for **execution**
- Writeback (~~in~~~~out~~ of program order):
 - When done the FU puts its results on the Result Bus which allows the RUU and the LSQ to be updated – the instr completes
- Commit (~~in~~~~out~~~~of~~ program order):
 - When appropriate, **commit** the instr's result data to the state locations (i.e., update D\$ and RegFile)

Baseline Superscalar MIPS Processor Model

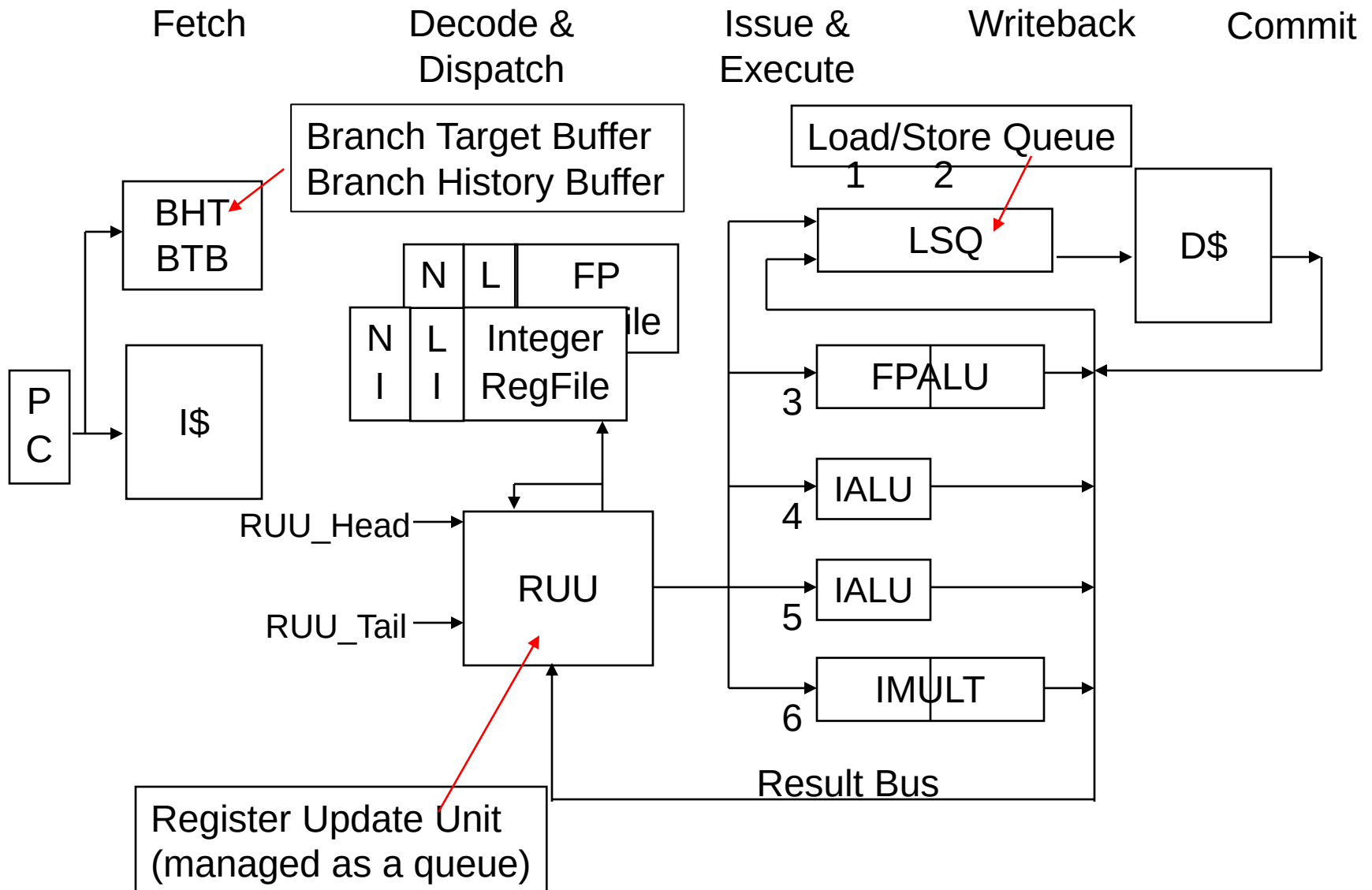


Fig. 9 from Sohi's paper

Today's outline

- Review:
 - Basic pipeline
 - Issue & Completion
 - Data Dependency
- SuperScalar
 - From Basic Pipeline to SS
 - RUU (function, example)
 - LSQ

Register Update Unit (RUU)

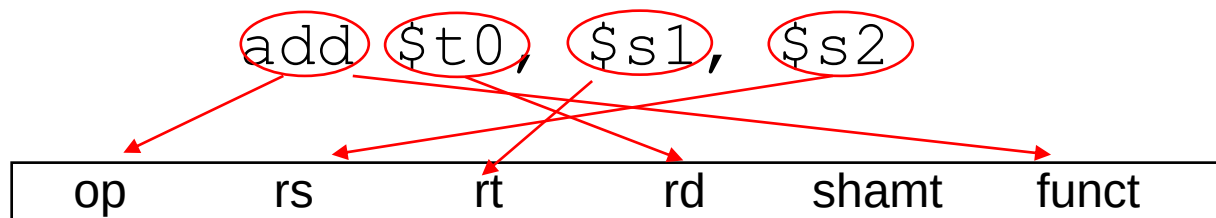
- What is RUU?
 - A **hardware** data structure that is used to resolve data dependencies by keeping track of an instruction's data and execution needs and that commits completed instructions in program order.

Major Functions of the RUU

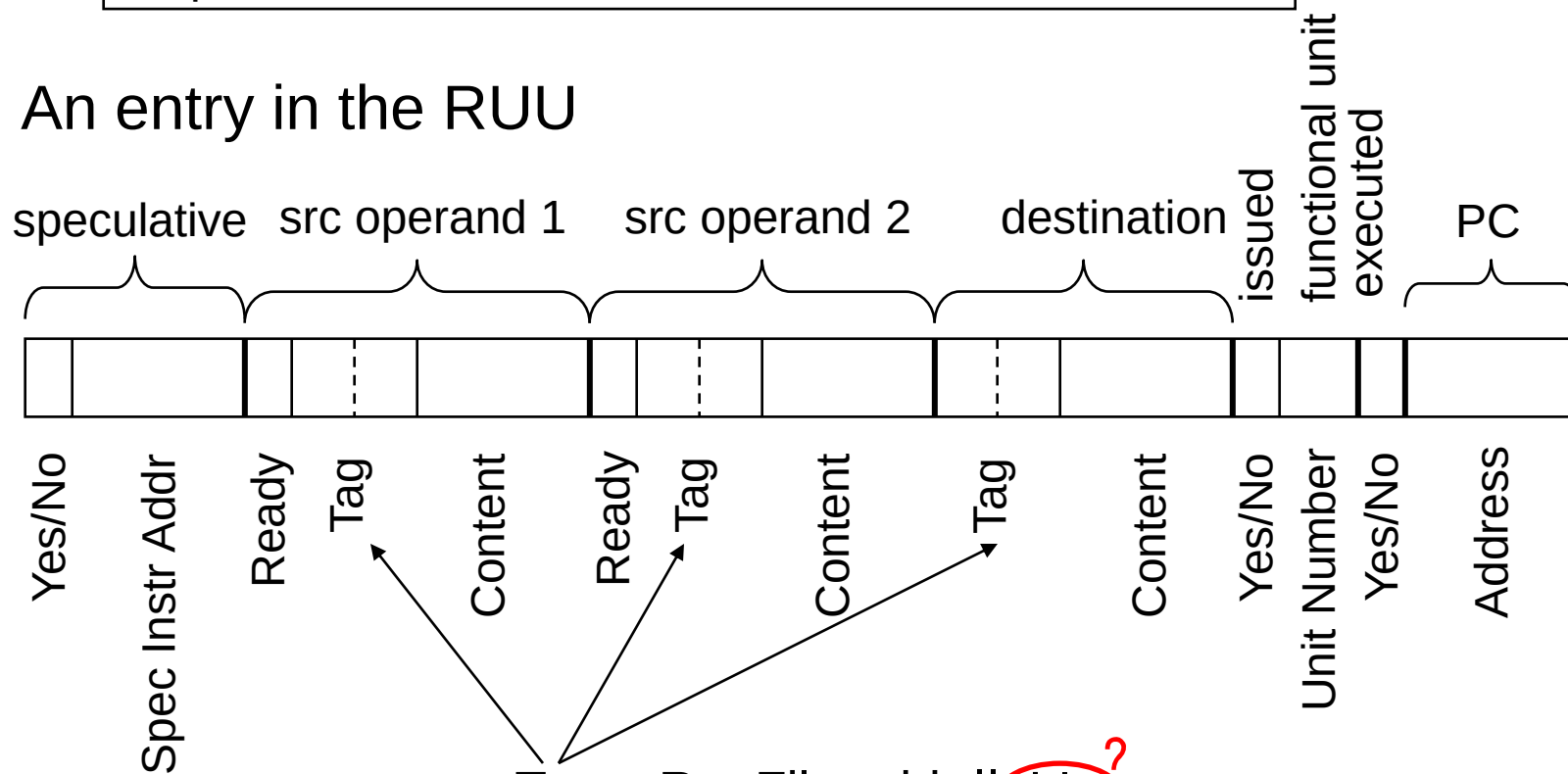
- Each of the tasks are done in parallel every cycle
- 1. Accepts new instructions from the Decode & Dispatch logic
- 2. Monitors the Result Bus to resolve true dependencies and to do write back of result data to the RUU
- 3. Determines which instructions are ready for execution, reserves the Result Bus, and issues the instruction to the appropriate FU
- 4. Determines if an instruction can commit (i.e., change the machine state) and commits the instruction if appropriate

Entry in RUU

- The basic Instr (an R format as example)



- An entry in the RUU

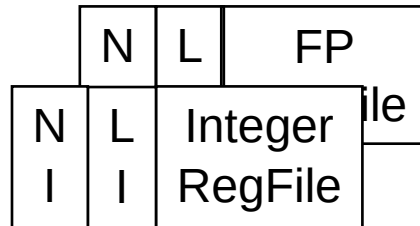


Tag = RegFile addr || LI

Fig. 10 in Sohi's paper

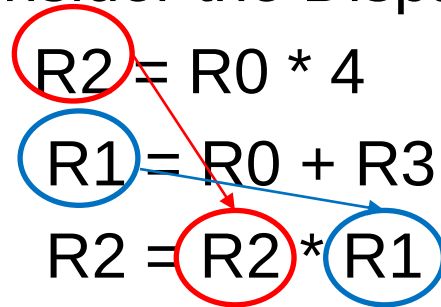
Additional RegFile Fields

- Each register in the general purpose RegFile has two associated n-bit counters (n of 3 is typical)
 - NI (**number of instances**): the number of instances of a register as a **destination register** in the RUU
 - LI (**latest instance**): the index of the latest instance
- When an instruction with destination register address R_i is dispatched to the RUU, both its NI and LI are incremented
 - Dispatch is blocked if a destination register's NI is $2^n - 1$, so only up to $2^n - 1$ instances of a register can be present in the RUU at any one time



Additional RegFile Fields

Example : Just consider the Dispatch Process!

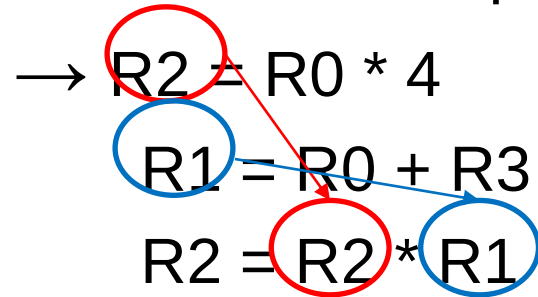
$$\begin{aligned} R2 &= R0 * 4 \\ R1 &= R0 + R3 \\ R2 &= R2 * R1 \end{aligned}$$


Initial	NI	LI	Value
\$0	0	0	0
\$1	0	0	3
\$2	0	0	?
\$3	0	0	2

Additional RegFile Fields

Example : Just consider the Dispatch Process!

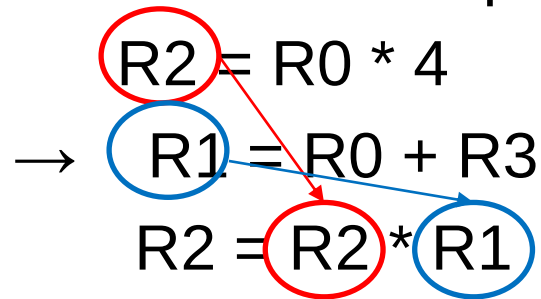
→ $R2 = R0 * 4$
 $R1 = R0 + R3$
 $R2 = R2 * R1$



	NI	LI	Value
\$0	0	0	0
\$1	0	0	3
\$2	1	1	?
\$3	0	0	2

Additional RegFile Fields

Example : Just consider the Dispatch Process!

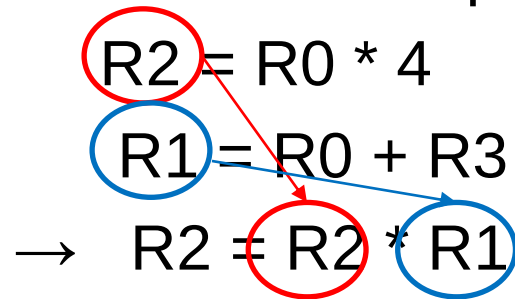


	NI	LI	Value
\$0	0	0	0
\$1	1	1	3
\$2	1	1	?
\$3	0	0	2

Additional RegFile Fields

Example : Just consider the Dispatch Process!

$R2 = R0 * 4$
 $R1 = R0 + R3$
 $\rightarrow R2 = R2 * R1$



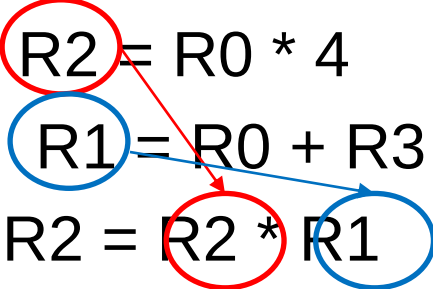
	NI	LI	Value
\$0	0	0	0
\$1	1	1	3
\$2	2	2	?
\$3	0	0	2

Additional RegFile Fields

- When an instruction is committed (updates the R_i value) the associated NI is decremented
 - When $NI = 0$ the register is “free” (there are no instruction in the RUU that are going to write to that register) and LI is cleared
 - This would be explained together with RUU.

Additional RegFile Fields

Example : Just consider the Commit Process!

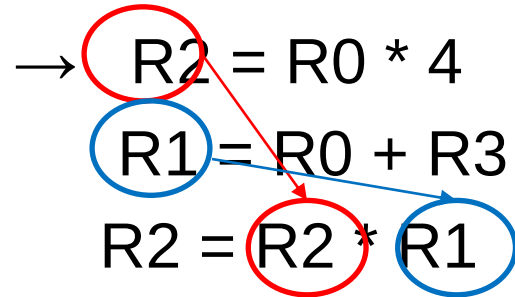
$$\begin{aligned} R2 &= R0 * 4 \\ R1 &= R0 + R3 \\ R2 &= R2 * R1 \end{aligned}$$


	NI	LI	Value
\$0	0	0	0
\$1	1	1	3
\$2	2	2	?
\$3	0	0	2

Additional RegFile Fields

Example : Just consider the Commit Process!

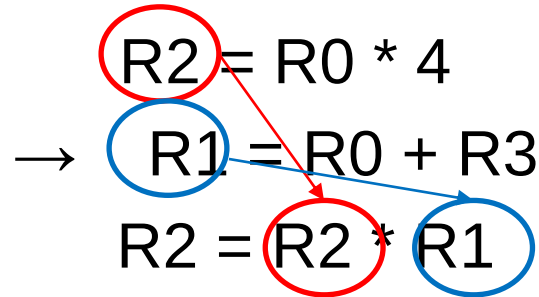
→ $R2 = R0 * 4$
 $R1 = R0 + R3$
 $R2 = R2 * R1$



	NI	LI	Value
\$0	0	0	0
\$1	1	1	3
\$2	1	2	0
\$3	0	0	2

Additional RegFile Fields

Example : Just consider the Commit Process!

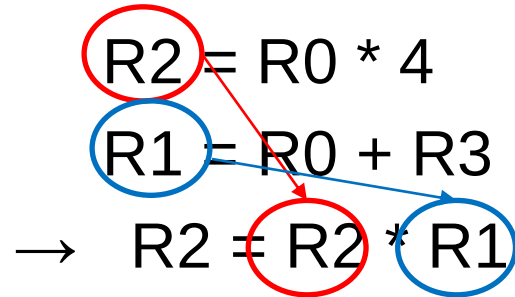


	NI	LI	Value
\$0	0	0	0
\$1	0	0	2
\$2	1	2	0
\$3	0	0	2

Additional RegFile Fields

Example : Just consider the Commit Process!

$R2 = R0 * 4$
 $R1 = R0 + R3$
 $\rightarrow R2 = R2 * R1$



	NI	LI	Value
\$0	0	0	0
\$1	0	0	2
\$2	0	0	0
\$3	0	0	2

Baseline Superscalar MIPS Processor Model

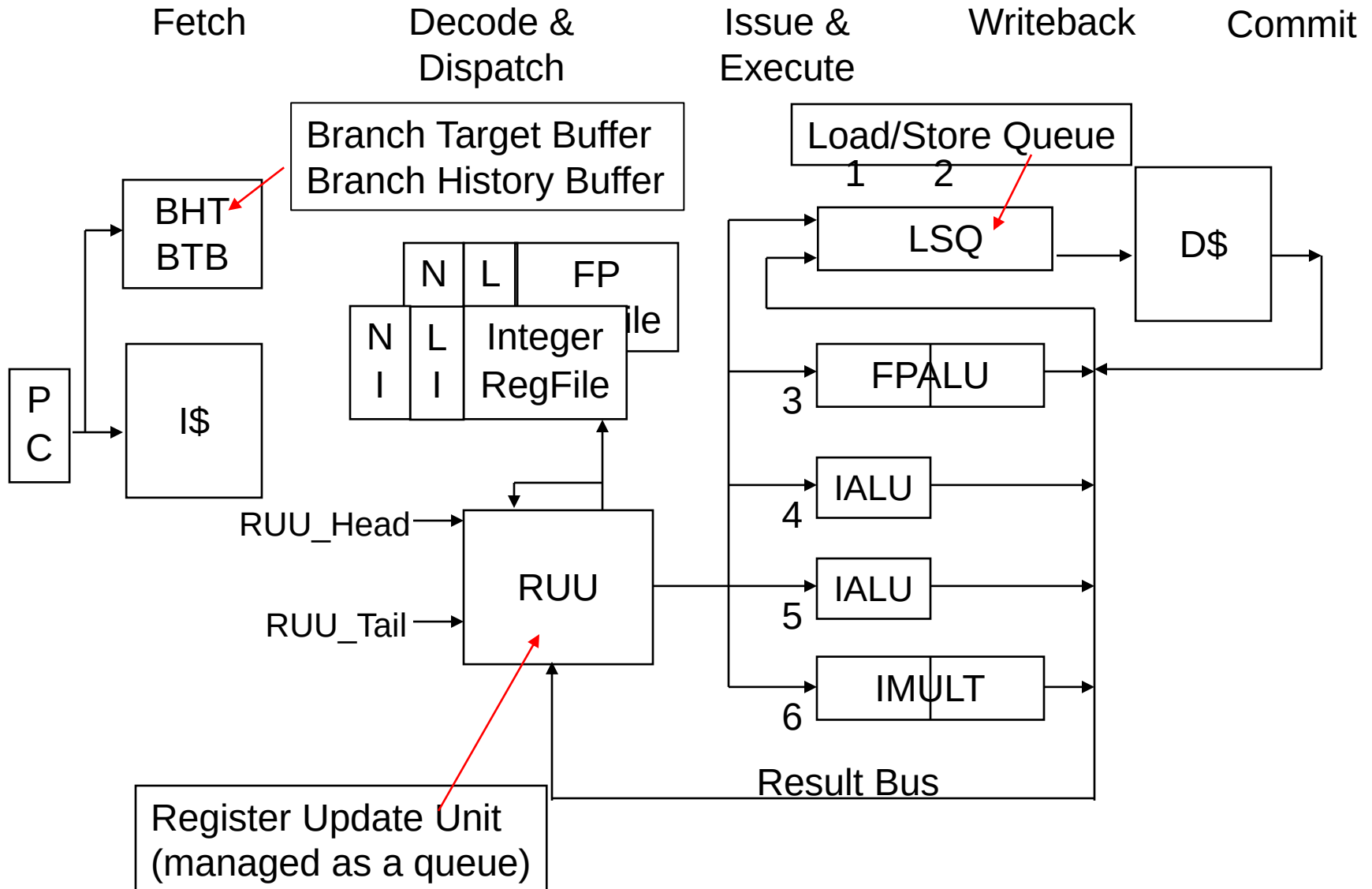
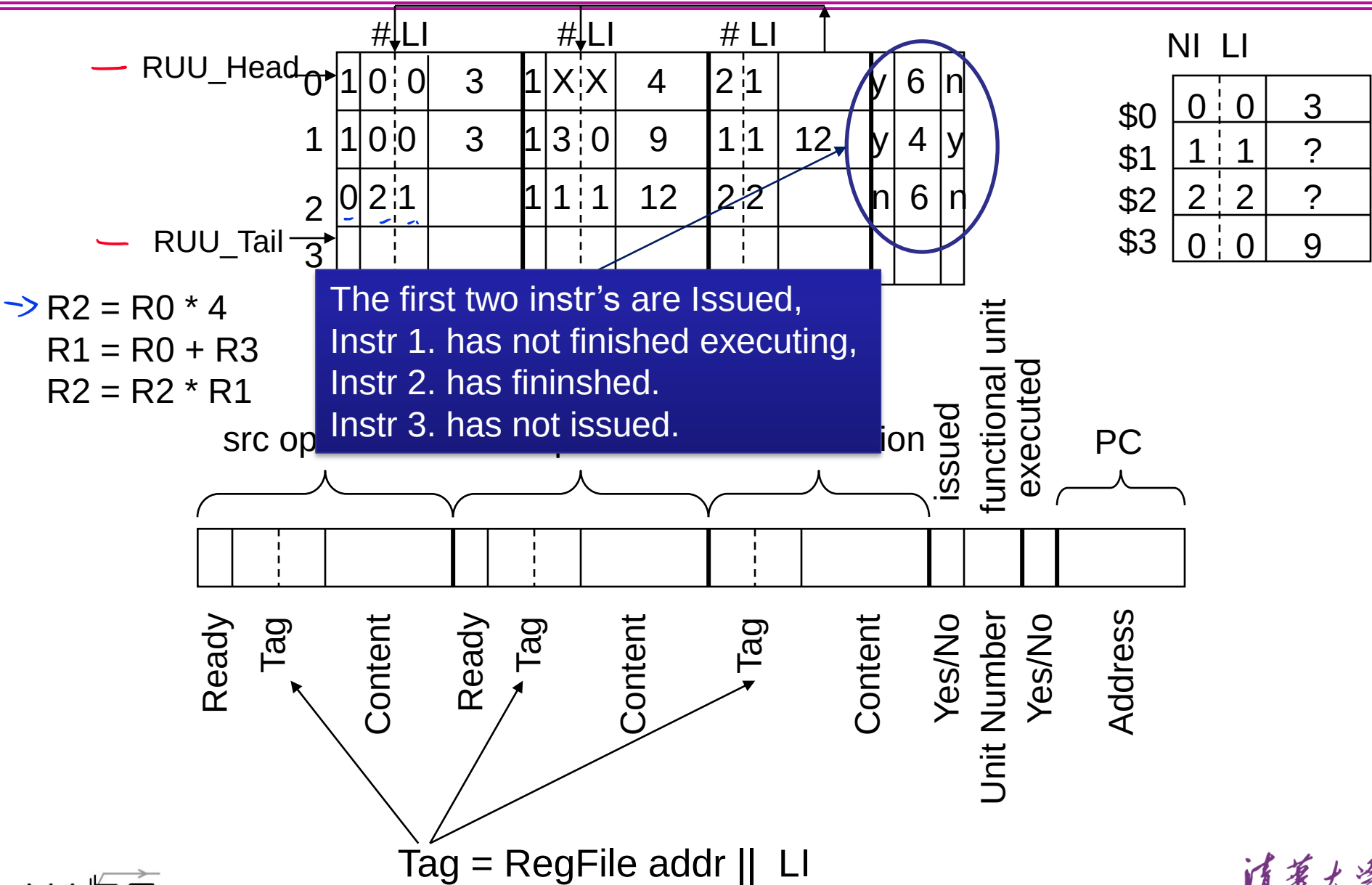


Fig. 9 from Sohi's paper

Register Update Unit (RUU) –Circular Queue

- We need to operate two pointer
 - RUU_head, RUU_tail
- When an Instr is dispatched into RUU
 - $\text{RUU_tail} \leftarrow \text{mod}(\text{RUU_tail} + 1, \text{RUU_size})$
- When an Instr is committed
 - $\text{RUU_head} \leftarrow \text{mod}(\text{RUU_head} + 1, \text{RUU_size})$
- When $\text{RUU_head} == \text{RUU_tail}$
 - The Queue is full and stop dispatching

Example: Analyze the State that the Diagram Shows



Major Functions of the RUU

- Each of the tasks are done in parallel every cycle
- 1. Accepts new instructions from the Decode & Dispatch logic
- 2. Monitors the Result Bus to resolve true dependencies and to do write back of result data to the RUU
- 3. Determines which instructions are ready for execution, reserves the Result Bus, and issues the instruction to the appropriate FU
- 4. Determines if an instruction can commit (i.e., change the machine state) and commits the instruction if appropriate

The First Function of the RUU

1. Accepts new instructions from the Decode & Dispatch logic – for each instruction in the fetch packet
 - The dispatch logic gets an entry in the RUU (a circular queue)
 - The RUU_Tail entry (currently empty) is allocated to the instruction and RUU_Tail is updated
 - Then if Ruu_Head = Ruu_Tail the RUU is full and further instruction fetch stalls until the RUU_Head advances (as a result of a commit)

The First Function of the RUU

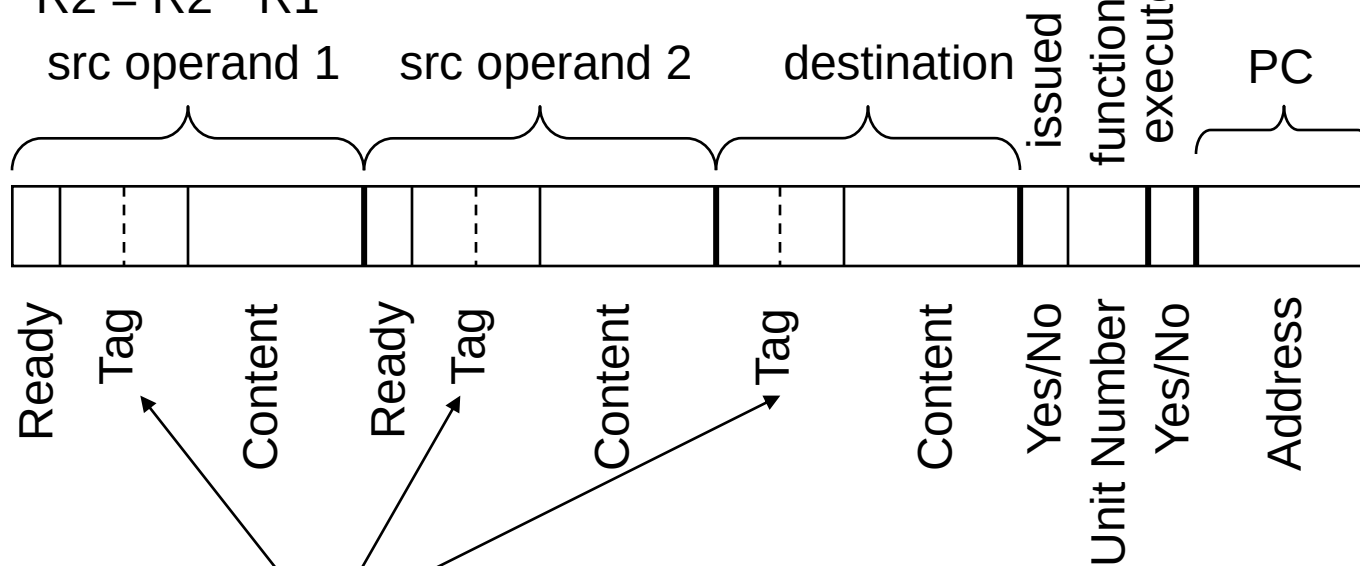
1. Accepts new instructions from the Decode & Dispatch logic – for each instruction in the fetch packet
 - For each **source operand**,
 - If the contents of the source register is **available**, then it is **copied** to the **Source_Content** field of the RUU entry and its **Ready_Bit** is **set**.
 - If **not**, the source **RegFile addr || LI** is **copied** to the source Tag field and the **Ready_Bit** is **reset**.
 - For the **destination operand**, the **RegFile destination addr || LI** is copied to the allocated **RUU destination Tag field**
 - The issued bit and executed bit are set to **No**, the number of the FU needed for the operation is entered, and the PC address of the instruction is copied to the PC Address field

Register Update Unit (RUU) –Circular Queue

		#LI				#LI				#LI					
RUU_Head	0	1	0	0	3	1	X	X	4	2	1		y	6	r
	1	1	0	0	3	1	3	0	7	1	1		y	4	r
RUU_Tail	2														
	3														

	NI	LI	
\$0	0	0	3
\$1	1	1	?
\$2	1	1	?
\$3	0	0	7

Issue: R2 = R0 * 4
 Issue: R1 = R0 + R3
 R2 = R2 * R1



Tag = RegFile addr || LI

Register Update Unit (RUU) –Circular Queue

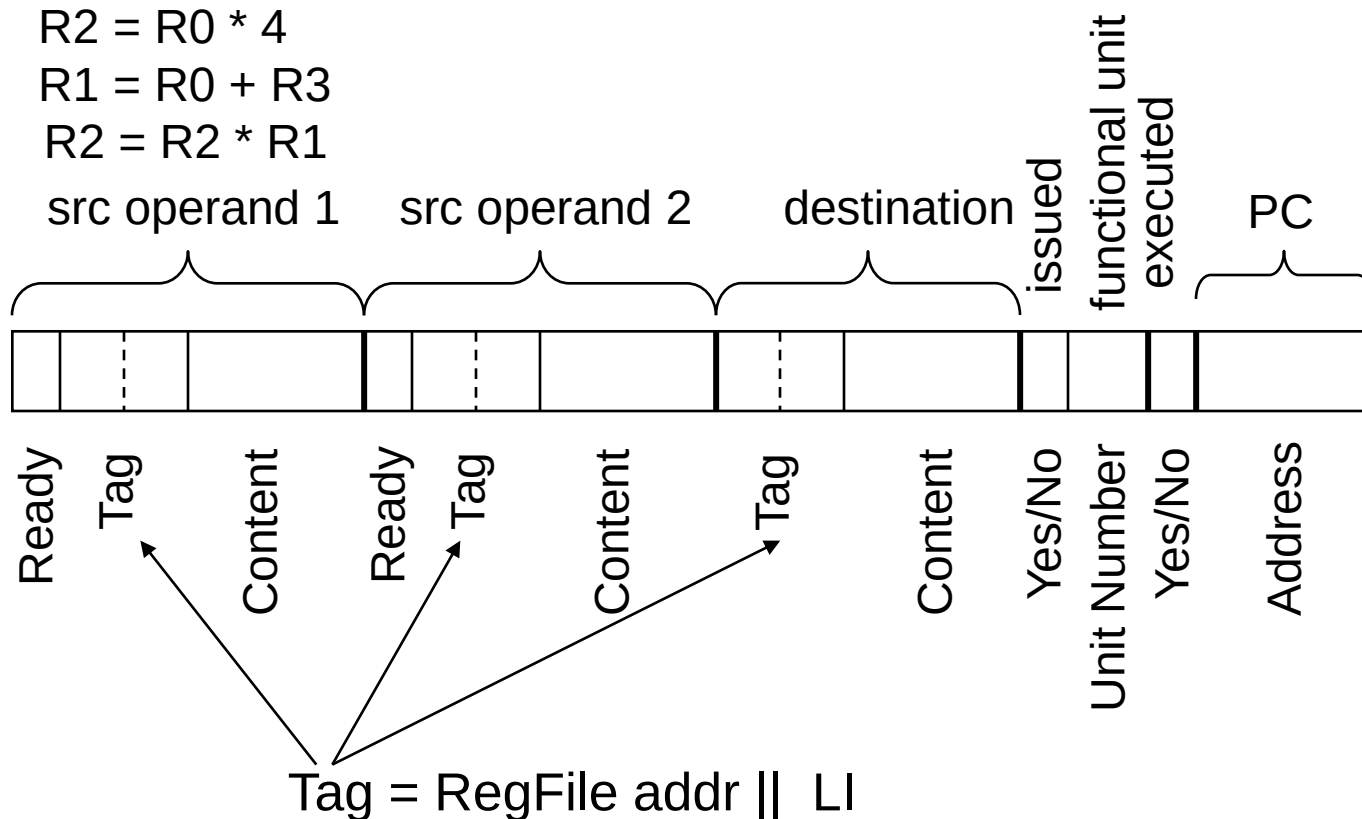
		#LI				#LI				#LI					
RUU_Head	0	1	0	0	3	1	X	X	4	2	1		y	6	r
	1	1	0	0	3	1	3	0	7	1	1		y	4	r
	2	0	2	1		0	1	1		2	2		n	6	r
RUU_Tail	3														

	NI	LI	
\$0	0	0	3
\$1	1	1	?
\$2	2	2	?
\$3	0	0	7

Issue: R2 = R0 * 4

Issue: R1 = R0 + R3

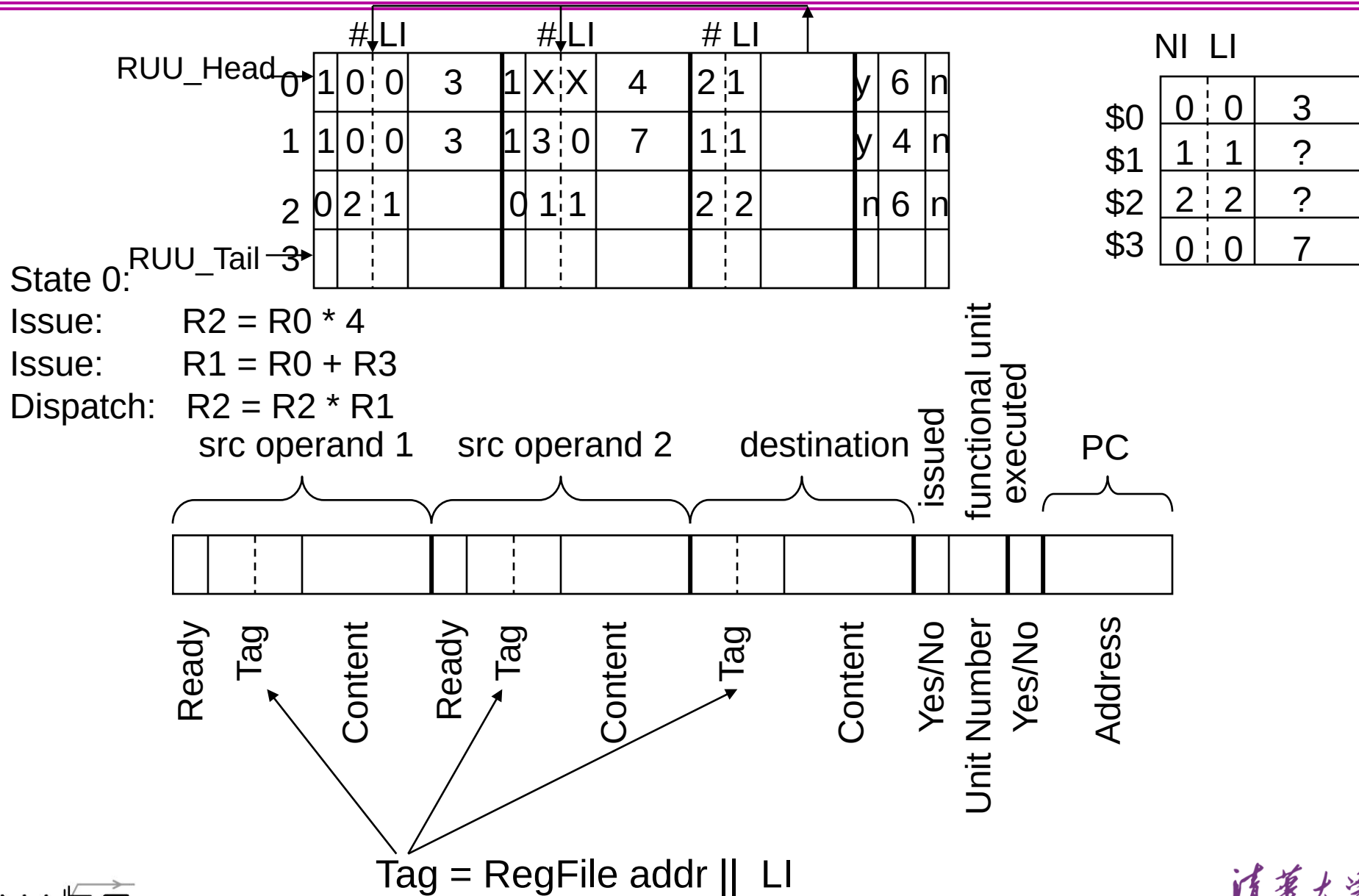
Dispatch: R2 = R2 * R1



The Second Function of the RUU

2. Monitors the Result Bus to resolve true dependencies and to do write back of result data to the RUU
 - The Result Bus destination addr || LI is compared **associatively** to the source Tag fields (for those source operands that are not Ready). If a match occurs, the data on the Result Bus is gated into the Content field for matching source operands.
 - The Result Bus contains the result data, its RUU entry address, and its RegFile destination addr || LI
 - The result data is gated into the destination Content field of the RUU entry that matches the RUU entry addr on the Result Bus
 - The executed bit is set to Yes

Register Update Unit (RUU) –Circular Queue



Register Update Unit (RUU) –Circular Queue

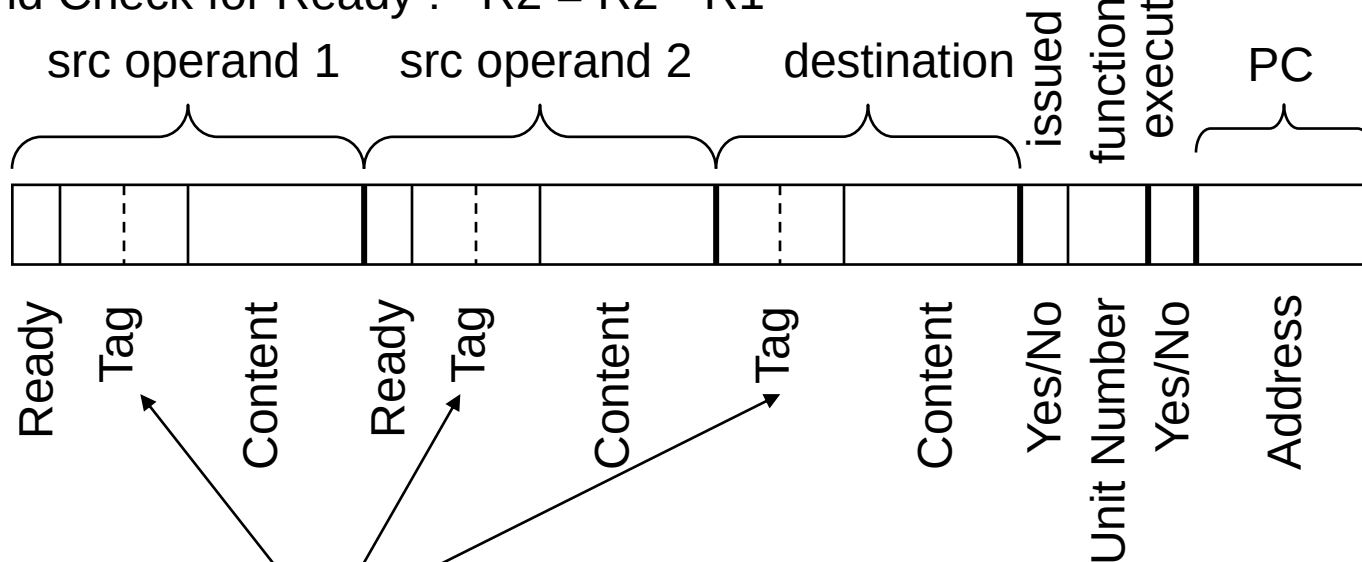
		# LI				# LI				# LI					
RUU_Head	0	1	0	0	3	1	X	X	4	2	1		y	6	n
	1	1	0	0	3	1	3	0	7	1	1	10	y	4	y
	2	0	2	1		1	1	1	10	2	2		n	6	n
State 1. RUU_Tail	3														

	NI	LI	
\$0	0	0	3
\$1	1	1	?
\$2	2	2	?
\$3	0	0	7

In Execution: $R2 = R0 * 4$

Finish Exe: $R1 = R0 + R3$

In RUU and Check for Ready : $R2 = R2 * R1$



Tag = RegFile addr || LI

The Second Function of the RUU

2. Monitors the Result Bus to resolve true dependencies and to do write back of result data to the RUU
 - Solves **anti-dependencies** through LI (making sure that the source fields get updated only for the *correct* version of the data)

The Third Function of the RUU

3. Determines which instructions are ready for execution, reserves the Result Bus, and issues the ready instructions to the appropriate FU's for execution
 - When **both** source operands of an RUU entry are **Ready**, the RUU issues the **highest priority** instruction – priority is given to load and store instr's and then to the instr's that entered the RUU the **earliest** (i.e., the ones **closest to RUU_Head**).
 - Reserves the Result Bus
 - Issues the instruction (sends to the **FU the source operands, RUU entry addr, and the destination's RegFile addr || LI**) and sets the issued bit to Yes
- Multiple instructions can be issued in parallel, if they are ready, if they can reserve the Result Bus, and if they are destined for different FU's

Register Update Unit (RUU) –Circular Queue

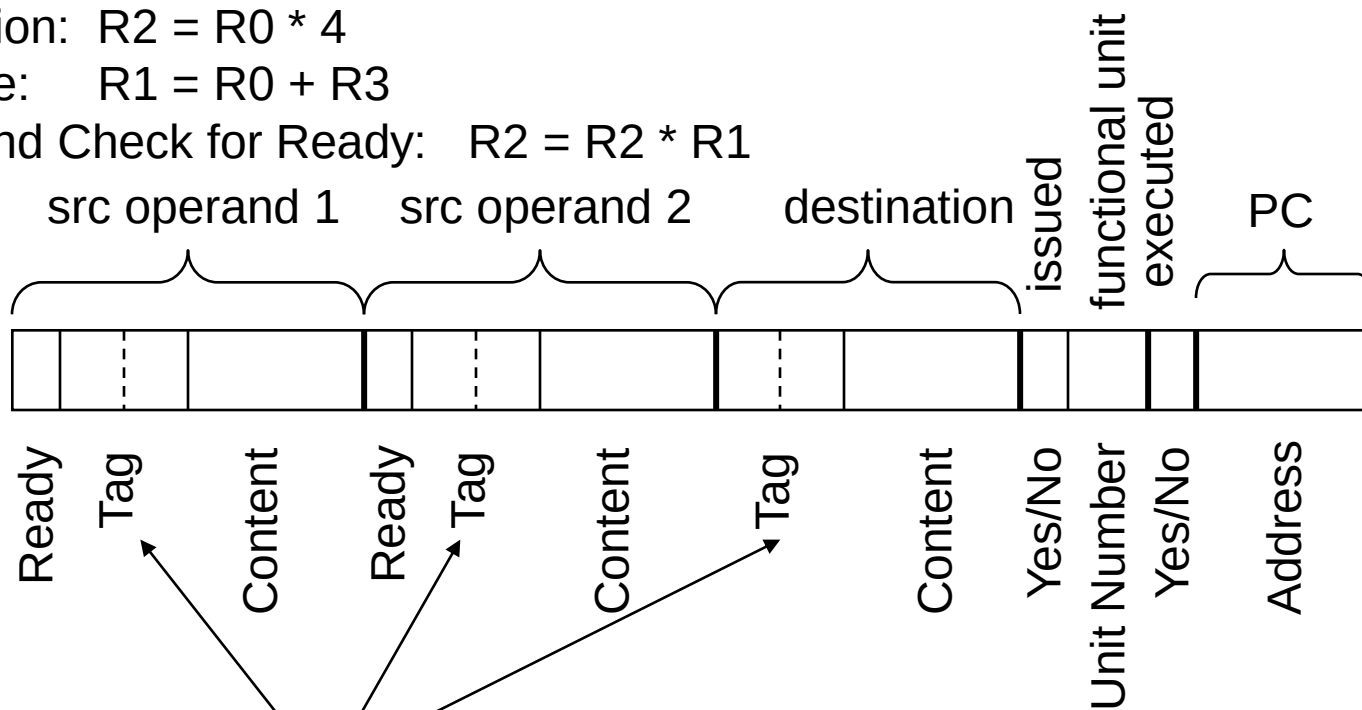
		# LI				# LI				# LI					
RUU_Head	0	1	0	0	3	1	X	X	4	2	1		y	6	n
	1	1	0	0	3	1	3	0	7	1	1	10	y	4	y
	2	0	2	1		1	1	1	10	2	2		n	6	n
State 1 RUU_Tail	3														

	NI	LI	
\$0	0	0	3
\$1	1	1	?
\$2	2	2	?
\$3	0	0	7

In Execution: $R2 = R0 * 4$

Finish Exe: $R1 = R0 + R3$

In RUU and Check for Ready: $R2 = R2 * R1$



Tag = RegFile addr || LI

Register Update Unit (RUU) –Circular Queue

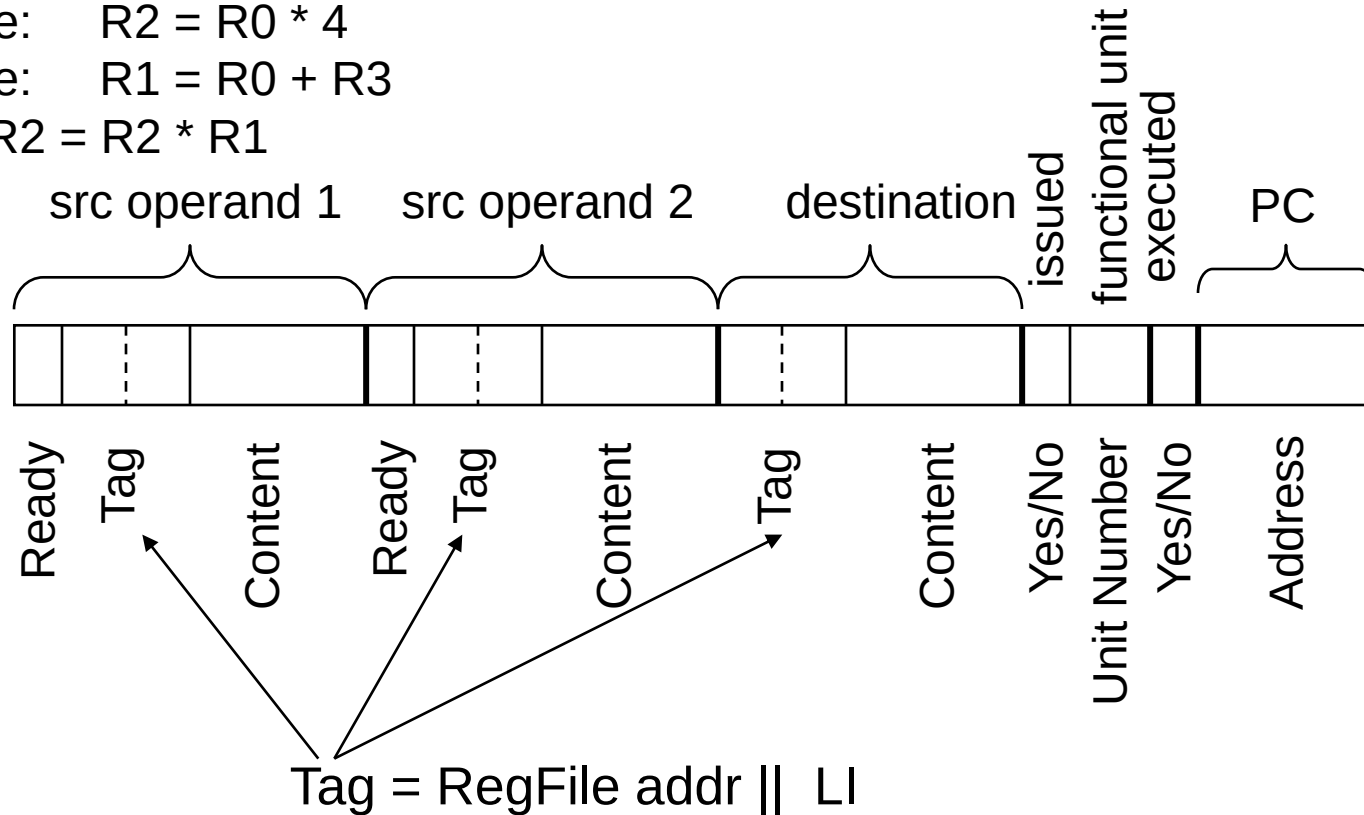
																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																											</
--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	----

	NI	LI	
\$0	0	0	3
\$1	1	1	?
\$2	2	2	?
\$3	0	0	7

Finish Exe: R2 = R0 * 4

Finish Exe: R1 = R0 + R3

In Exe: R2 = R2 * R1



The Third Function of the RUU

3. Determines which instructions are ready for execution, reserves the Result Bus, and issues the ready instructions to the appropriate FU's for execution
 - Resolves **true dependencies** through the Ready Bit (i.e., must wait for the source operands before issue)

The Fourth Function of the RUU

4. Determines if an instruction can **commit** (i.e., change the machine state) and commits the instruction if appropriate
 - Monitors the executed bit of the **RUU_Head** entry. If the bit is set, the destination content data is written into the RegFile at the destination's RegFile address
 - Matches the **destination RegFile addr || LI** against the RUU's **source Tag fields** and on match copies the destination content data into source Content fields
 - **Decrements** the associated RegFile entry's NI counter
 - **Releases the RUU** entry by incrementing the RUU_Head pointer
- ❑ **Solves output dependencies** by writing to RegFile in program order
- ❑ **Multiple instructions can commit** in parallel if they are ready to commit, if they are writing to different RegFile registers, and if there are multiple RegFile write ports

Register Update Unit (RUU) –Circular Queue

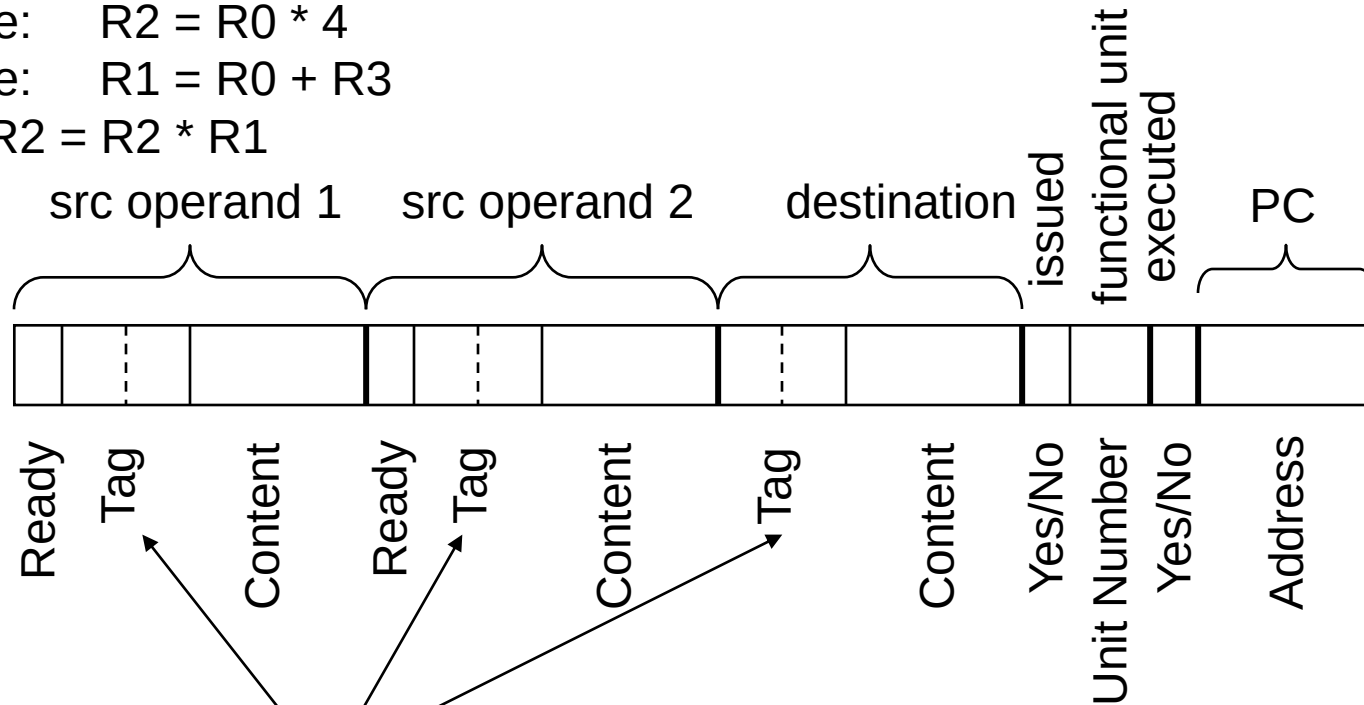
		# LI				# LI				# LI					
RUU_Head	0	1	0	0	3	1	X	X	4	2	1	12	y	6	y
	1	1	0	0	3	1	3	0	7	1	1	10	y	4	y
	2	1	2	1	12	1	1	1	10	2	2		y	6	r
State 2. RUU_Tail	3														

	NI	LI	
\$0	0	0	3
\$1	1	1	?
\$2	2	2	?
\$3	0	0	7

Finish Exe: R2 = R0 * 4

Finish Exe: R1 = R0 + R3

In Exe: R2 = R2 * R1



Tag = RegFile addr || LI

Register Update Unit (RUU) –Circular Queue

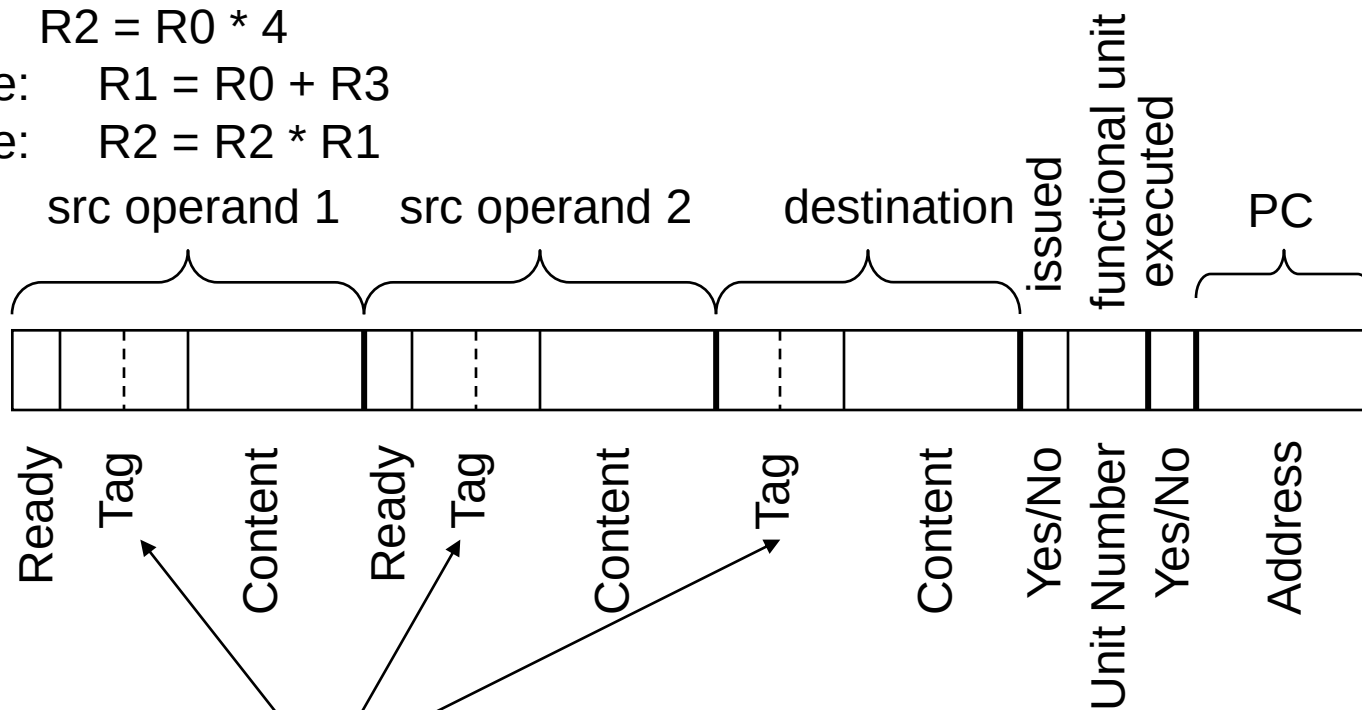
	#	LI		#	LI		#	LI				
0	1	0	0	3	1	X	X	4	2	1	12	y 6 y
RUU_Head → 1	1	0	0	3	1	3	0	7	1	1	10	y 4 y
2	1	2	1	12	1	1	1	10	2	2	120	y 6 y
State 3 RUU_Tail → 3												

	NI	LI	
\$0	0	0	3
\$1	1	1	?
\$2	1	2	12
\$3	0	0	7

Commit: $R2 = R0 * 4$

Finish Exe: $R1 = R0 + R3$

Finish Exe: $R2 = R2 * R1$



Register Update Unit (RUU) –Circular Queue

	# LI				# LI				# LI					
0	1	0	0	3	1	X	X	4	2	1	12	y	6	y
1	1	0	0	3	1	3	0	7	1	1	10	y	4	y
2	1	2	1	12	1	1	1	10	2	2	120	y	6	y
State 4.														

RUU_Head → 2

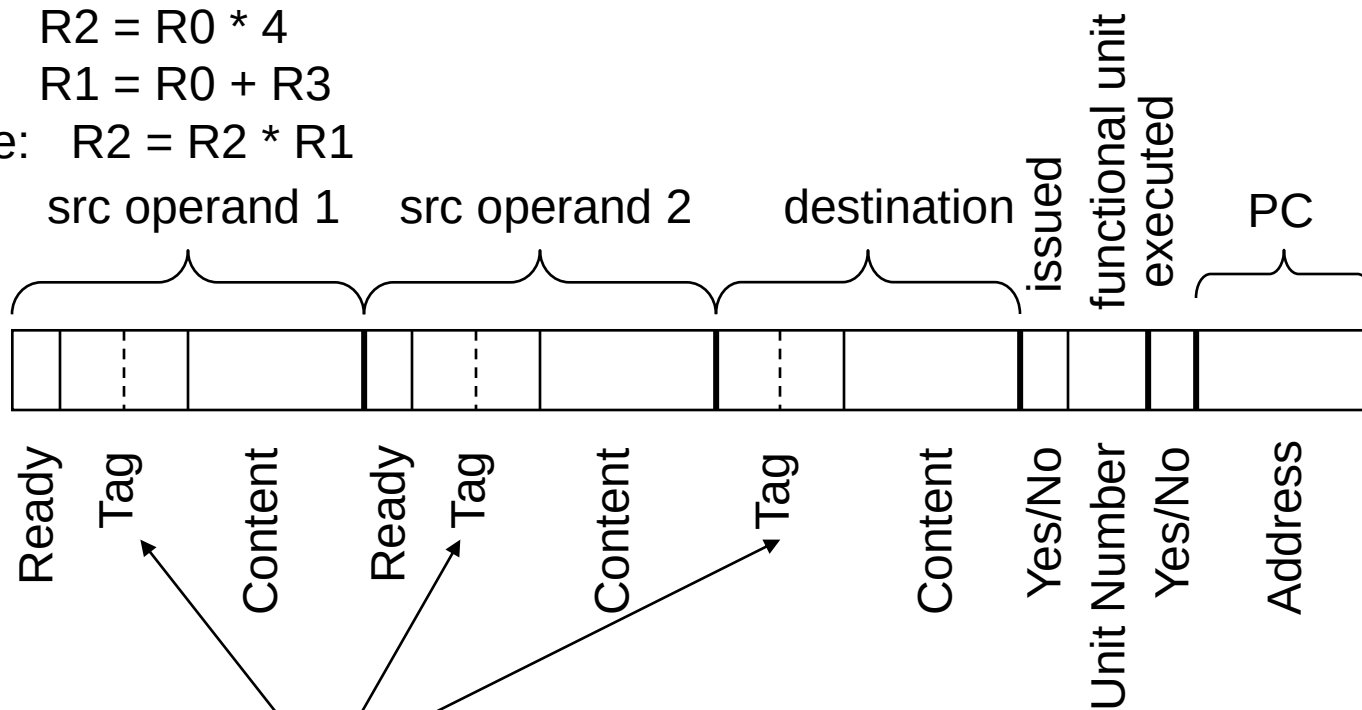
RUU_Tail → 3

	NI	LI	
\$0	0	0	3
\$1	0	0	10
\$2	1	2	12
\$3	0	0	7

Commit: $R2 = R0 * 4$

Commit: $R1 = R0 + R3$

Finish Exe: $R2 = R2 * R1$



Tag = RegFile addr || LI

Register Update Unit (RUU) –Circular Queue

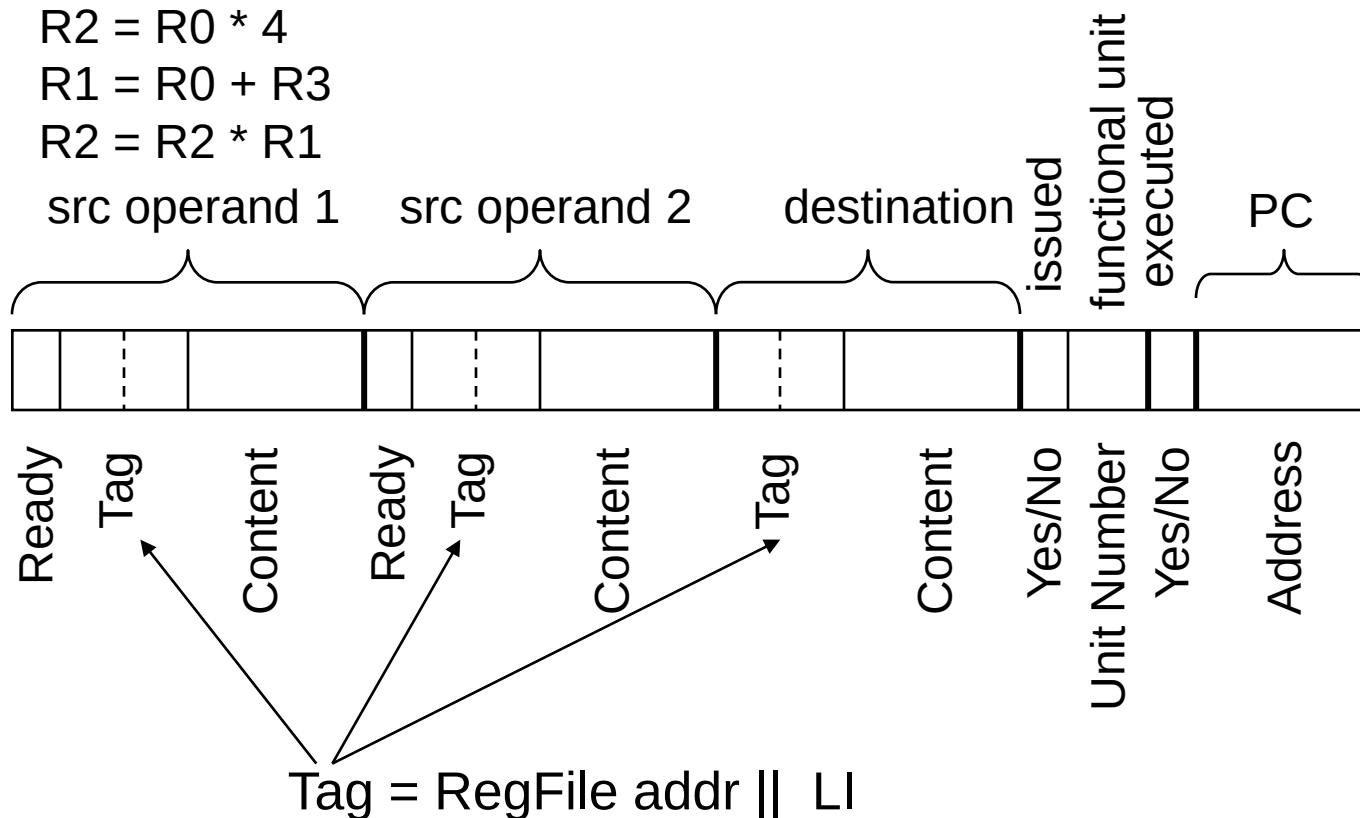
																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																</
--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	----

	NI	LI	
\$0	0	0	3
\$1	0	0	10
\$2	0	0	120
\$3	0	0	7

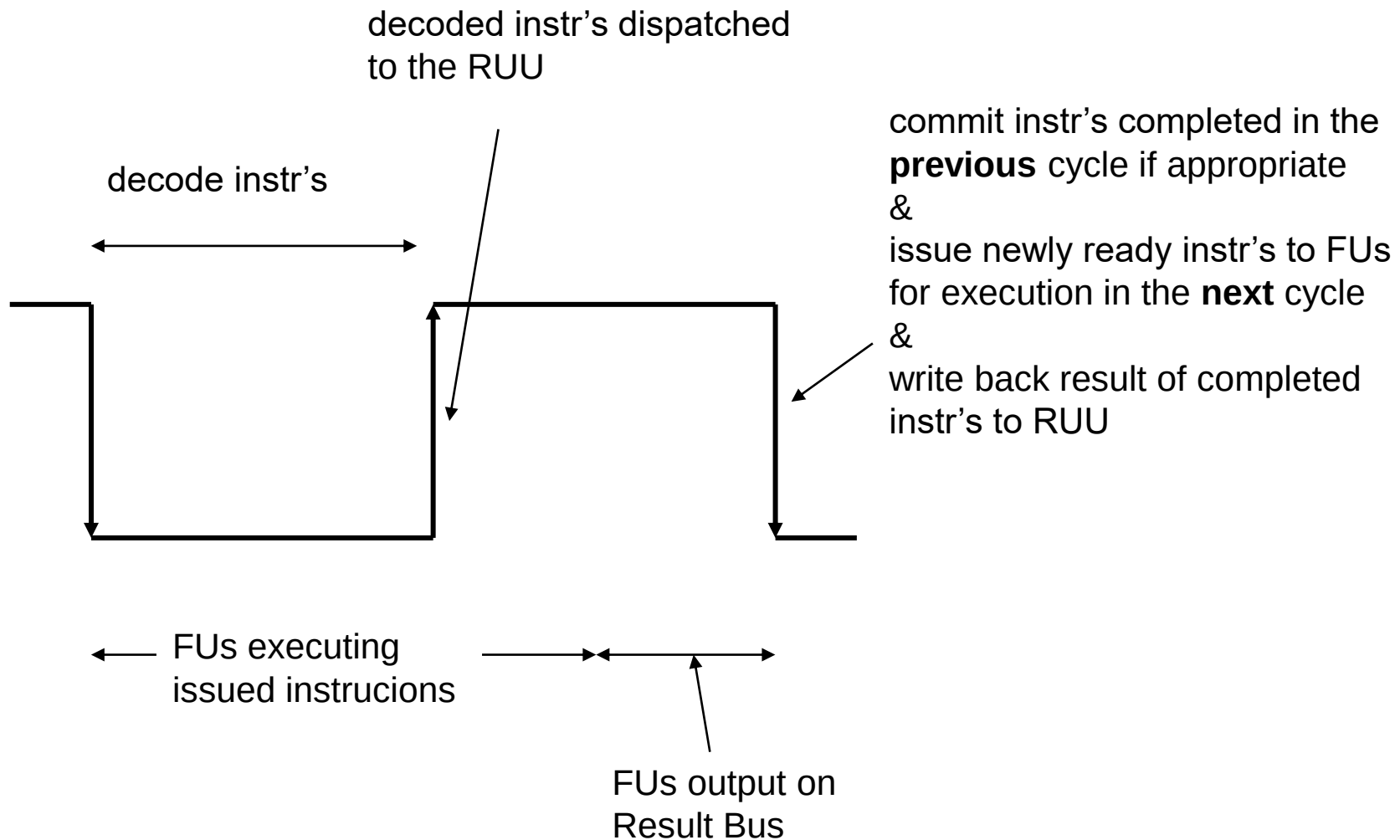
Commit: R2 = R0 * 4

Commit: R1 = R0 + R3

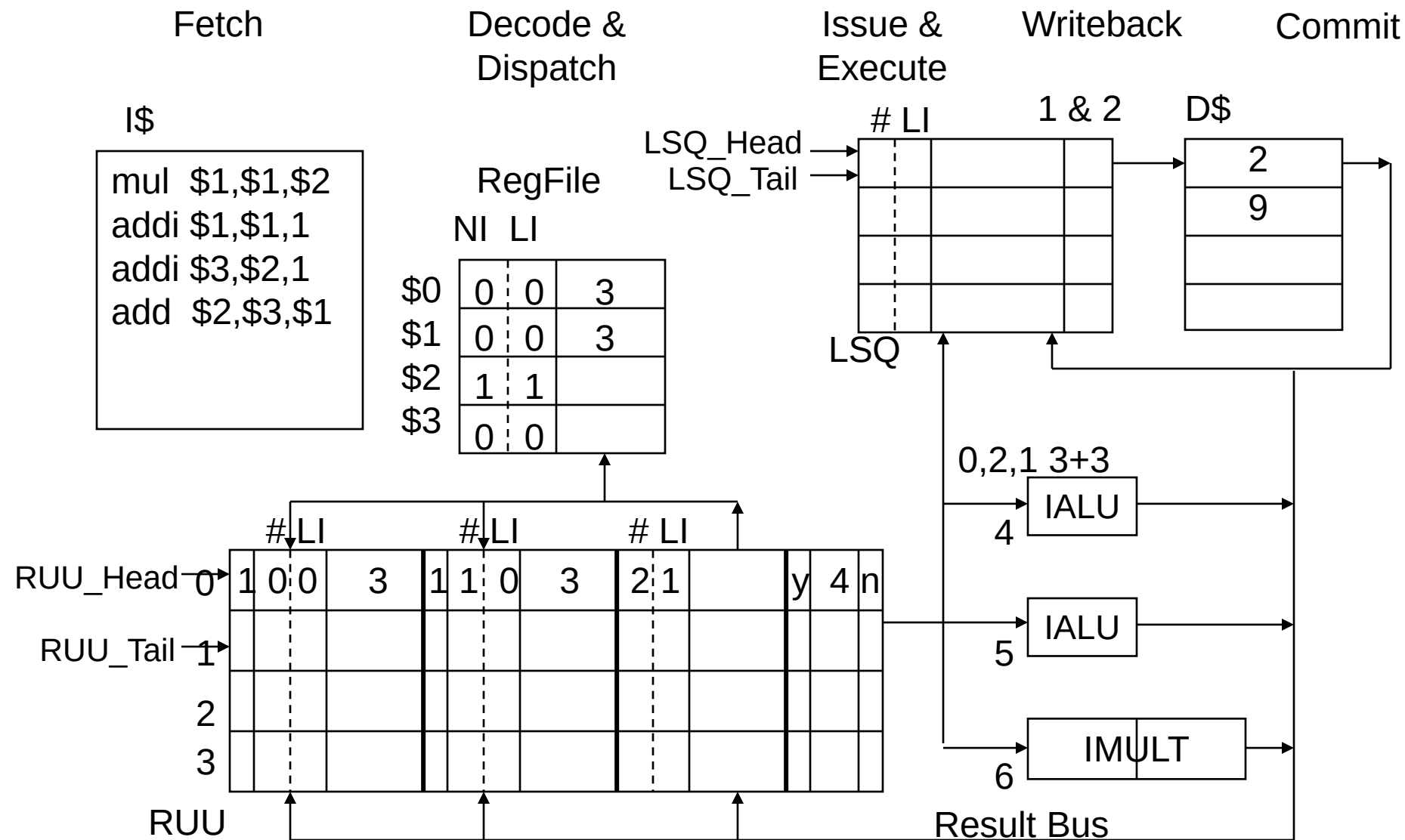
Commit: R2 = R2 * R1



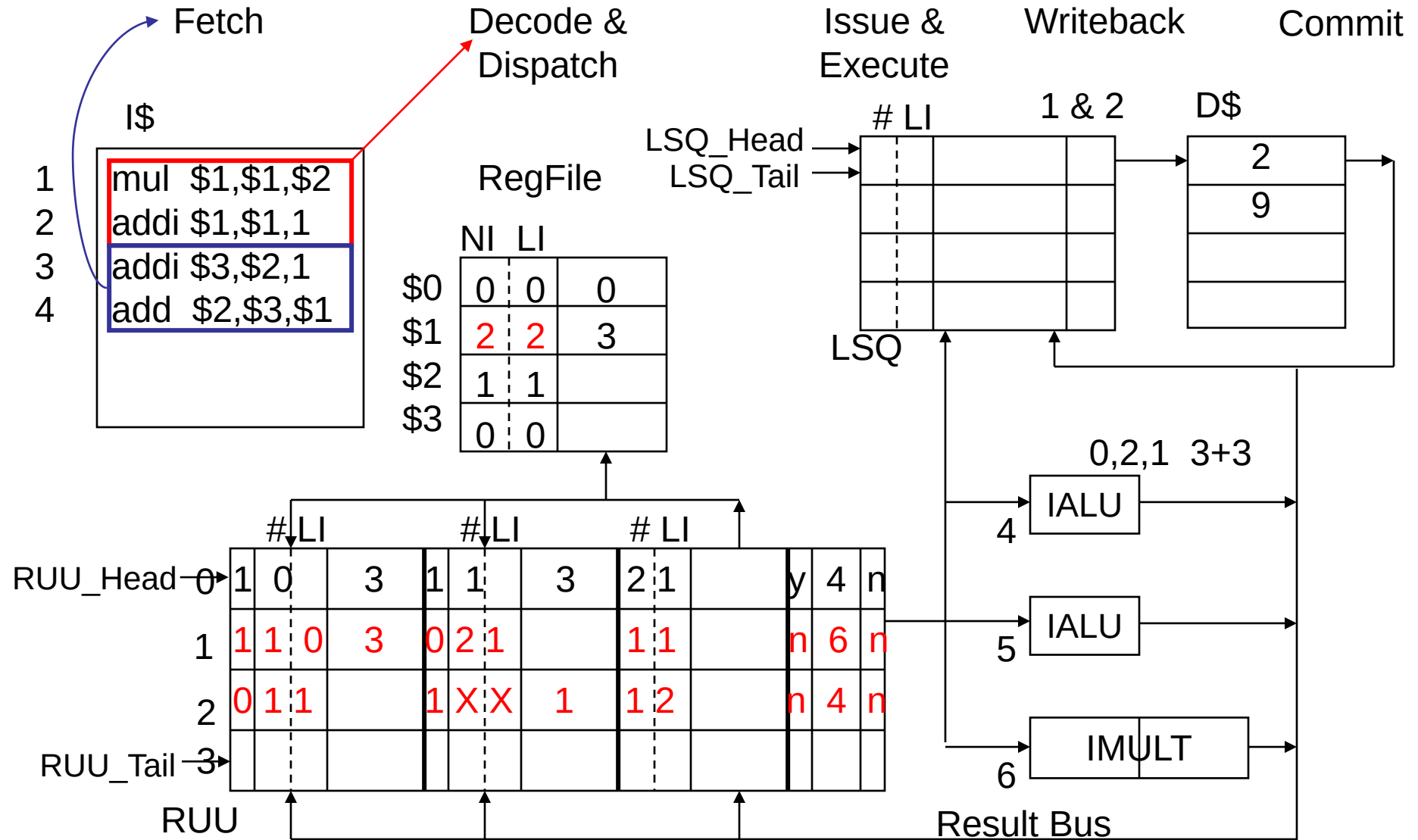
Timing of RUU Activities



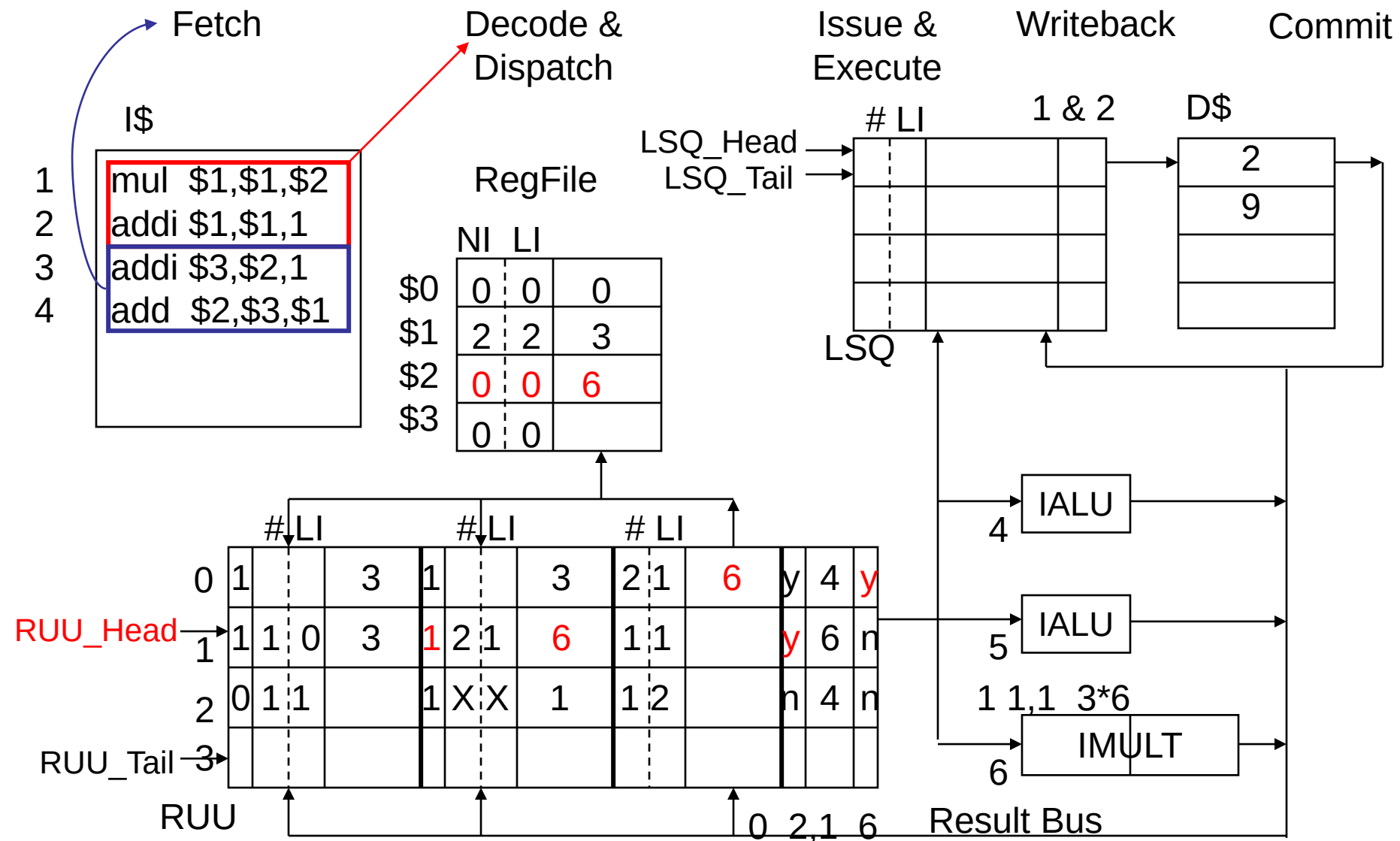
Baseline Superscalar MIPS Processor Model 0



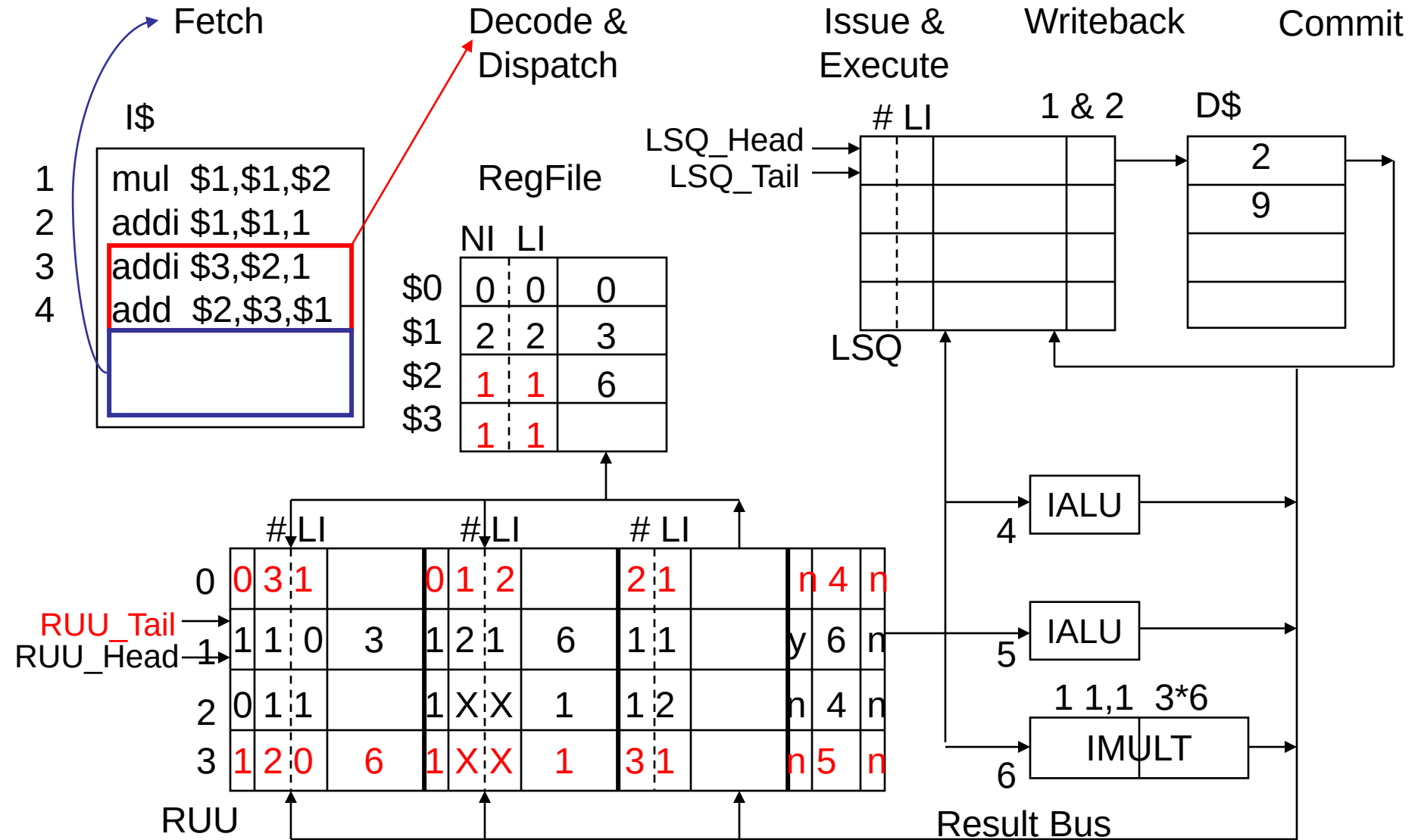
Baseline Superscalar MIPS Processor Model 1-1



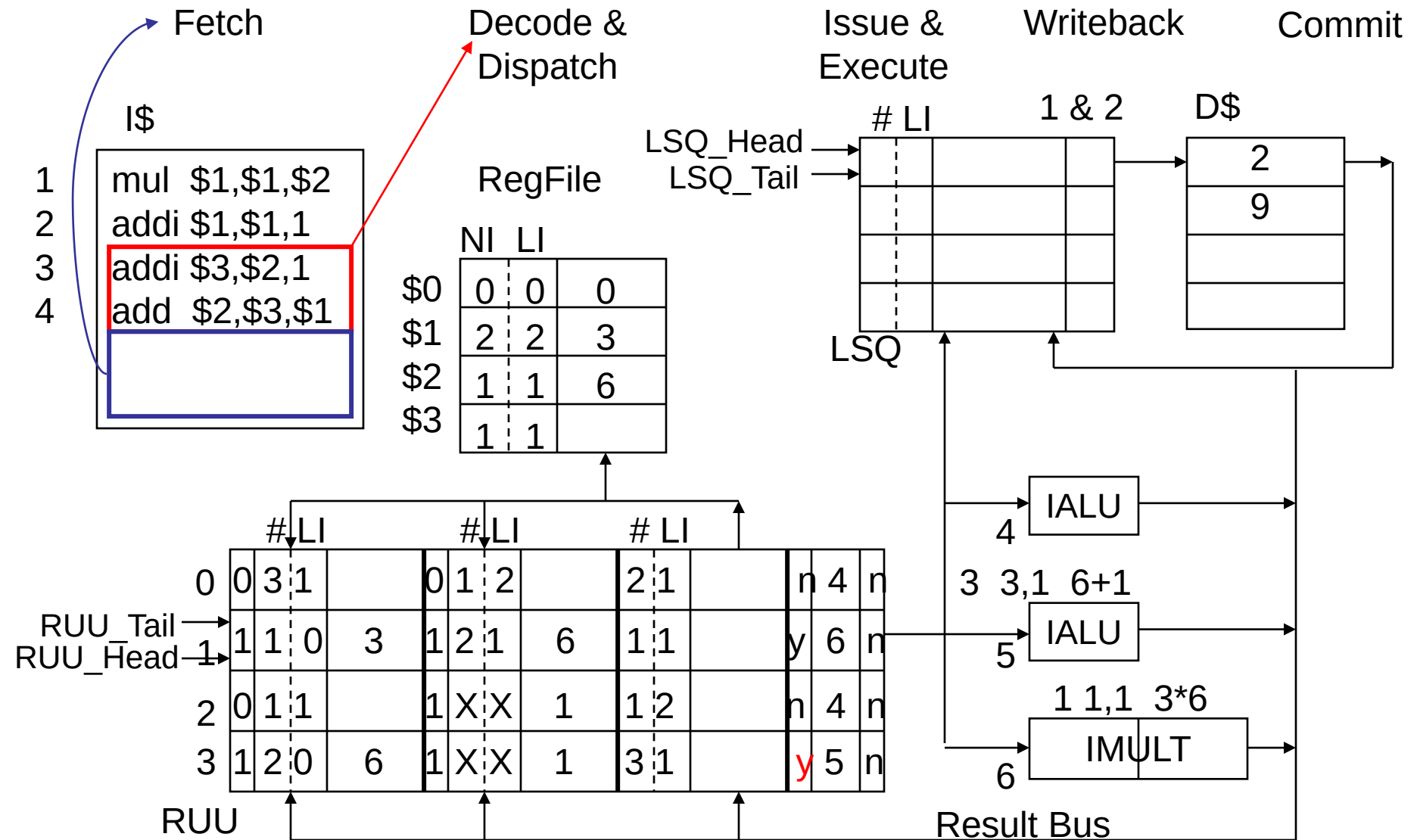
Baseline Superscalar MIPS Processor Model 1-2



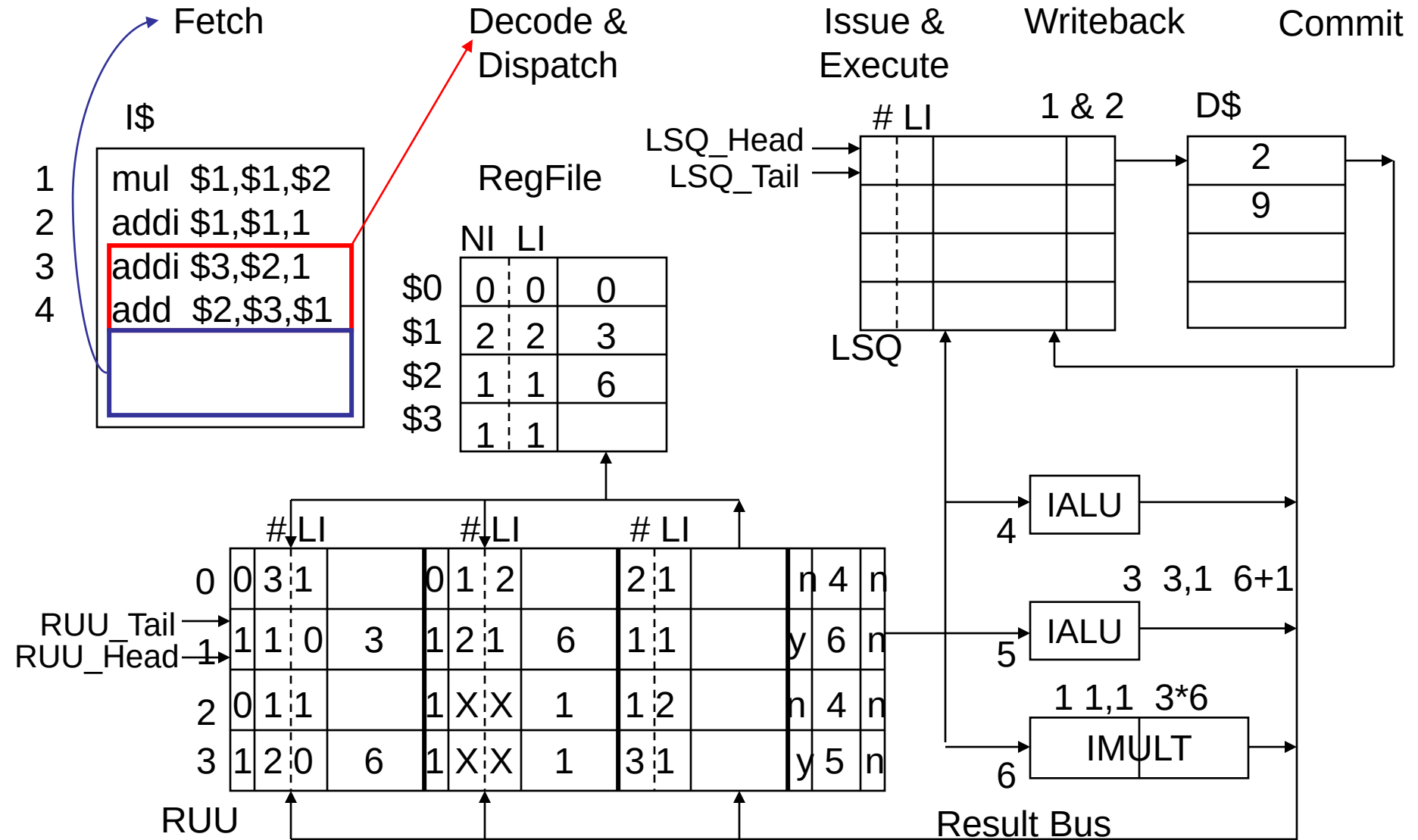
Baseline Superscalar MIPS Processor Model 2-1



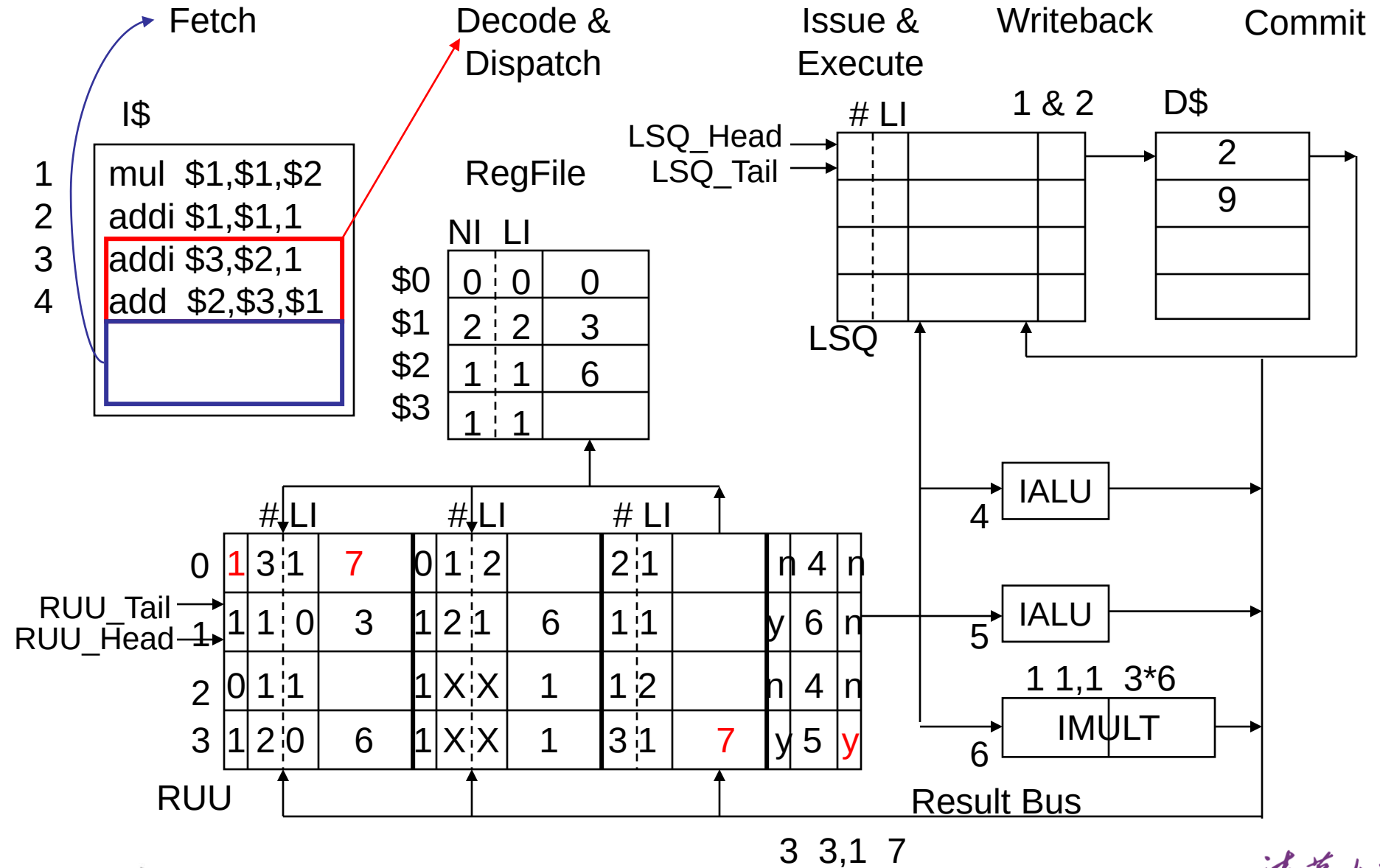
Baseline Superscalar MIPS Processor Model 2-2



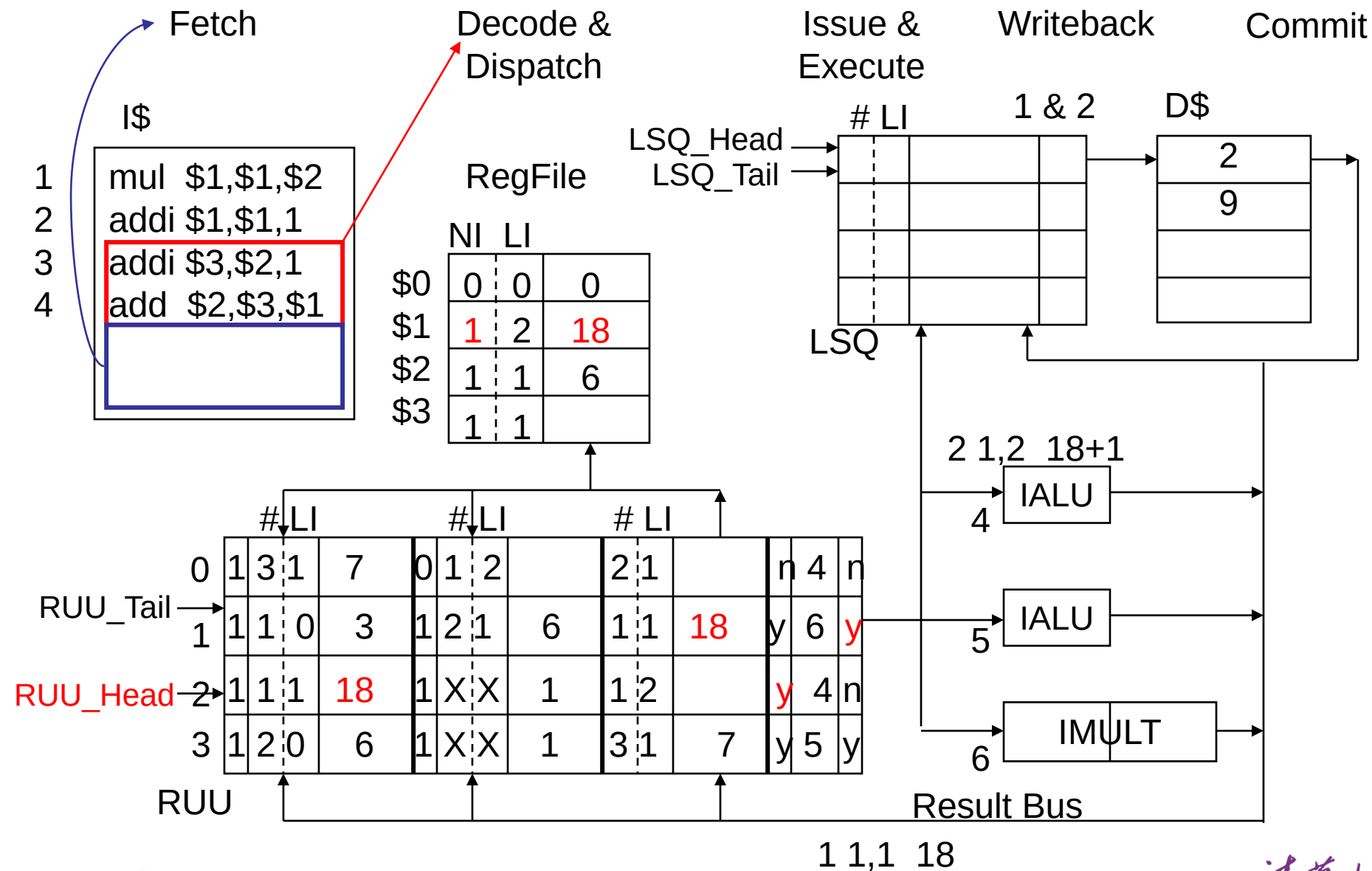
Baseline Superscalar MIPS Processor Model 3-1



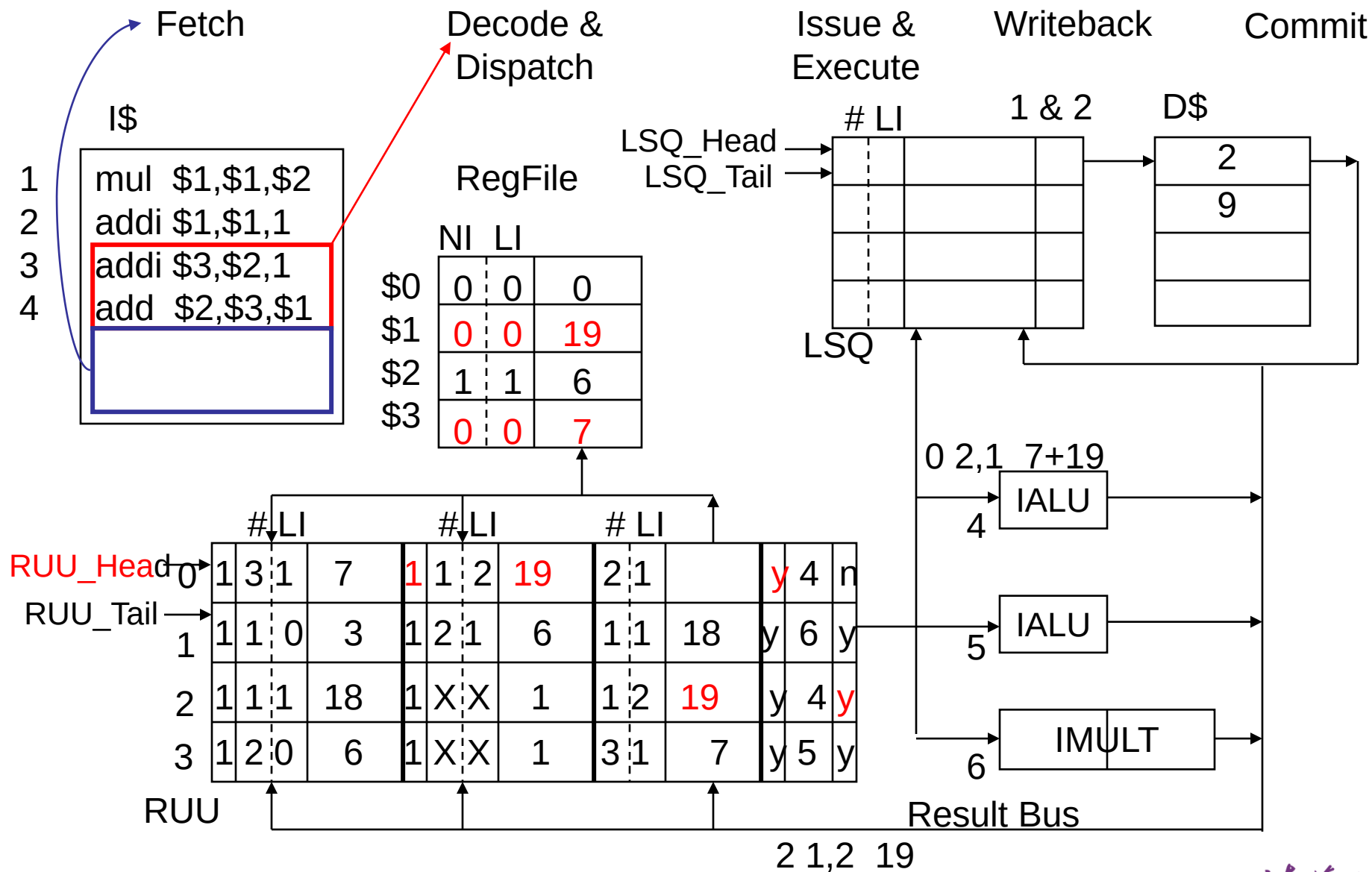
Baseline Superscalar MIPS Processor Model 3-2



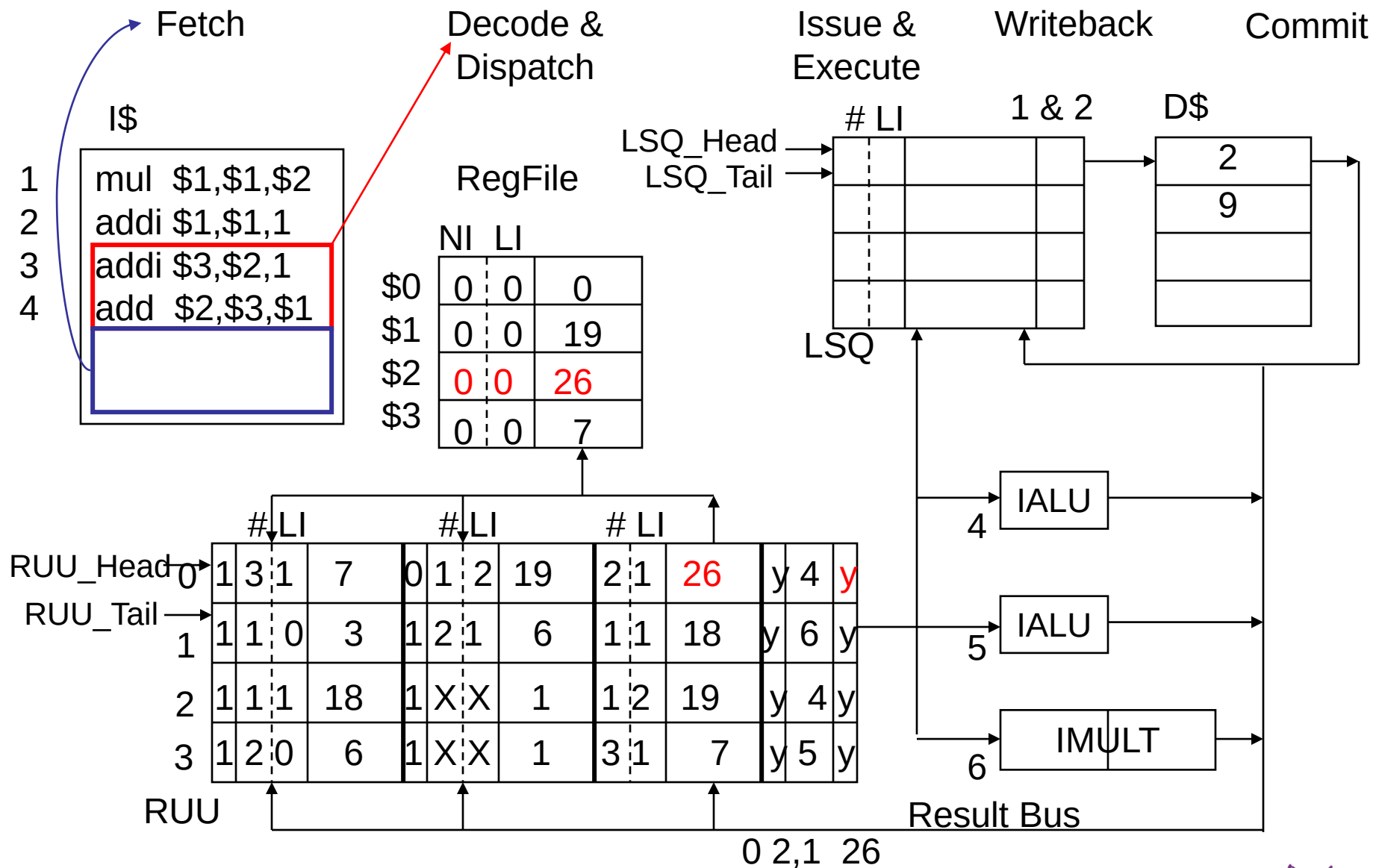
Baseline Superscalar MIPS Processor Model 4



Baseline Superscalar MIPS Processor Model 5



Baseline Superscalar MIPS Processor Model 6



RUU with Bypass Logic

- Consider an instruction that is *to be* added to the RUU whose source operand(s) have already been computed (i.e., exist somewhere in the RUU) but which have not yet been committed to the RegFile
 - The source Tag field comparison logic must also monitor the RUU to RegFile bus (in addition to the Result Bus)

□ With **result bypass logic**, can expedite the number of instr's ready to execute by doing an associative comparison of source operand Tags with the dst Tag of each RUU entry on instr Dispatch

From Sohi, 1990

# of RUU entries	Without bypass logic		With bypass logic	
	Speed up	Issue rate	Speed up	Issue rate
4	0.912	0.407	0.940	0.419
8	1.082	0.483	1.248	0.557
15	1.225	0.546	1.584	0.706
30	1.393	0.621	1.700	0.758

Summary

- **Question to answer after this Part:**
 - In RUU, how do we solve the three dependencies?
 - True data dependence?
 - Anti-dependence?
 - Output dependence?

Today's outline

- Review:
 - Basic pipeline
 - Issue & Completion
 - Data Dependency
- SuperScalar
 - From Basic Pipeline to SS
 - RUU (function, example)
 - LSQ

Baseline Superscalar MIPS Processor Model

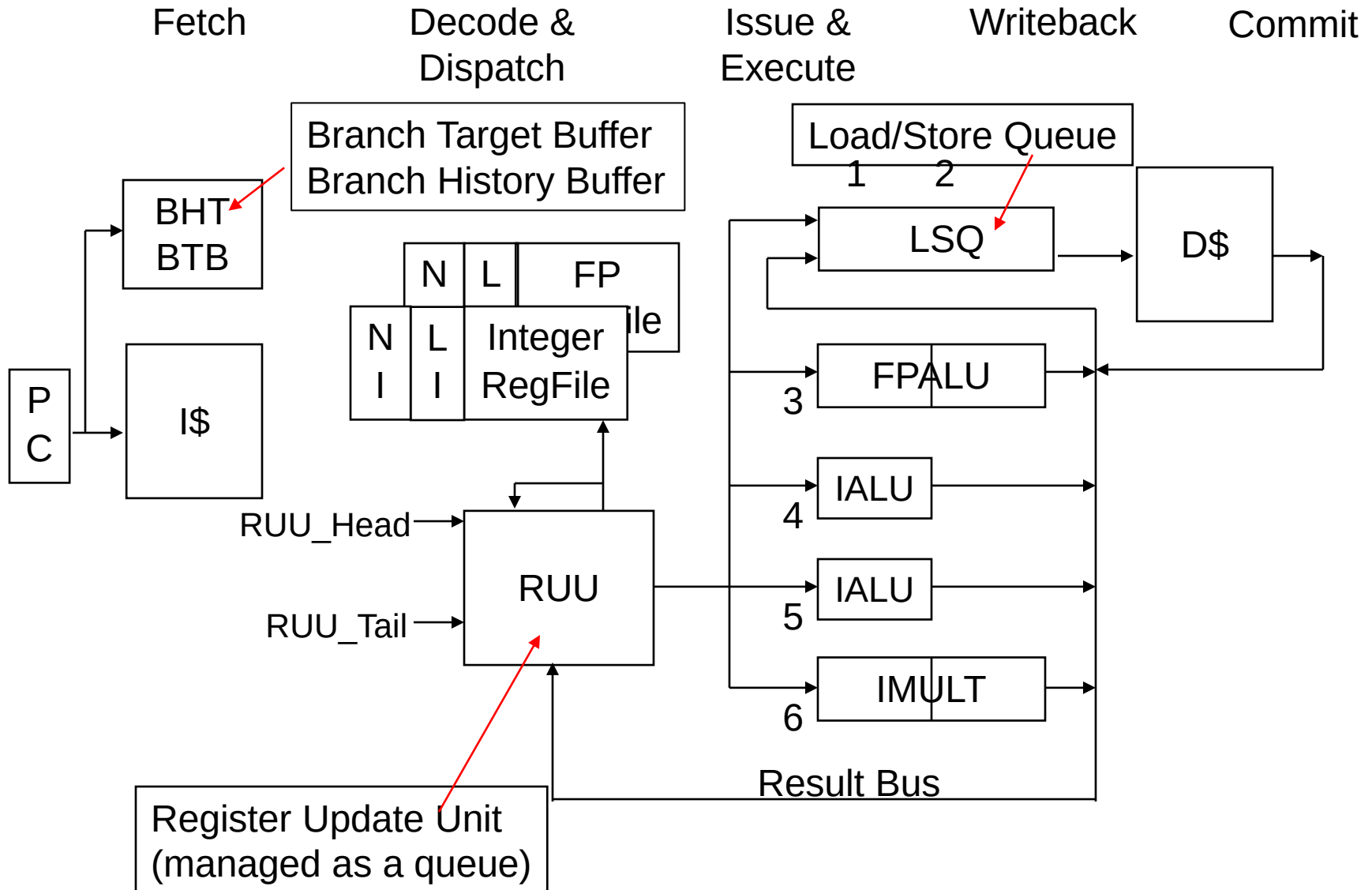


Fig. 9 from Sohi's paper

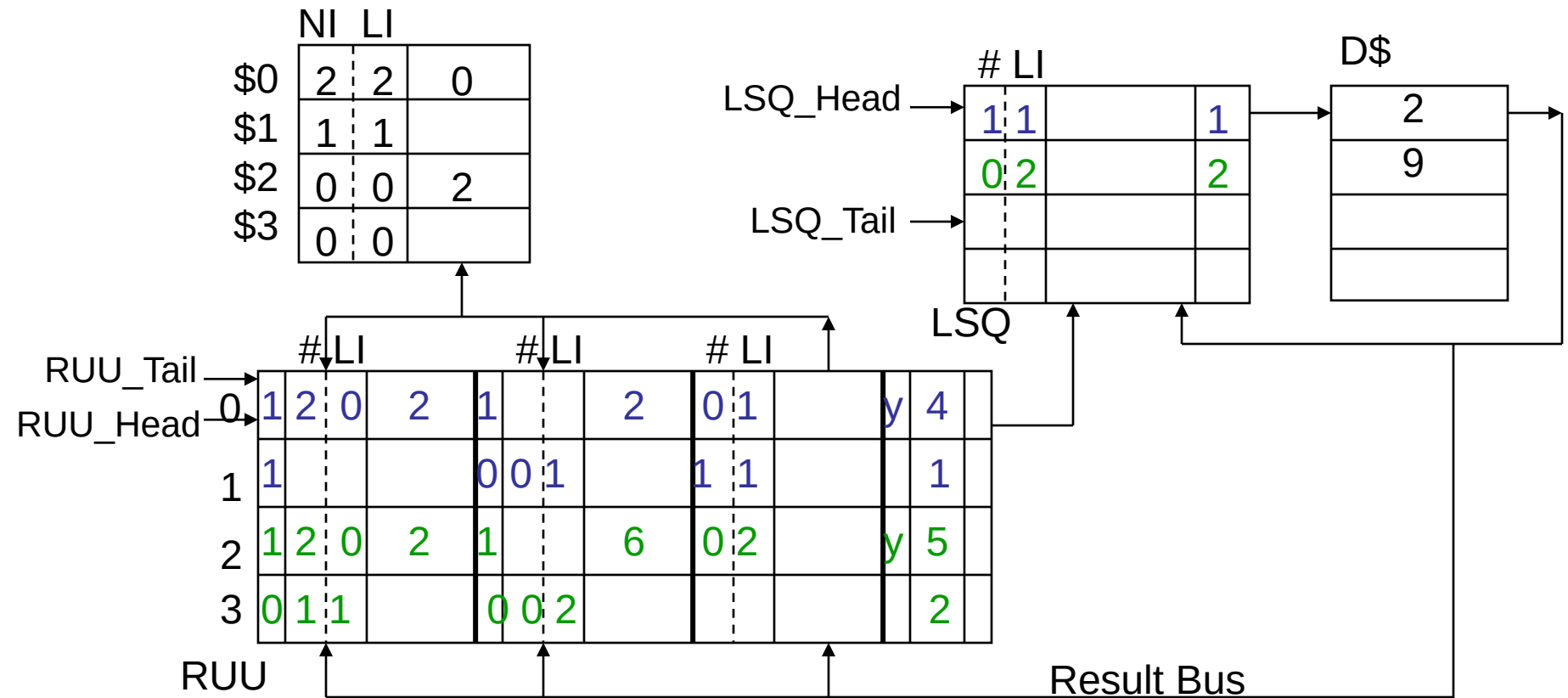
MicroOperations of Load and Store

- Recall that loads and stores are dispatched to the RUU as two (micro)instr's – one to compute the effective addr and one to do the memory operation
 - Load `lw R1, 2(R2)` becomes
 - `addi R0, R2, 2`
 - `lw R1, R0`
 - Store `sw R1, 6(R2)` becomes
 - `addi R0, R2, 6`
 - `sw R1, R0`
- At the same time a LSQ entry is allocated
 - Each LSQ entry consists of a Tag field (RegFile addr || LI) and a Content field. The LI counter allows for multiple instances of stores (writes) to a memory address
 - When a load completes (the D\$ returns the data on the Result Bus) or a store commits (in program order) the LSQ entry is released
 - Instruction dispatch is blocked if there is not a free LSQ entry and two free RUU entries

Load & Store RUU & LSQ Operation

Load
addi R0,R2,2
lw R1,R0

Store
addi R0,R2,6
sw R1,R0



Memory Location Data Dependencies

- We can have true, anti- and output dependencies on memory locations as well
 - Memory storage conflicts are infrequent since memory locations are not reused in the same way that registers are and since the number of true dependencies is small (loads and stores are less than 30% of the instruction mix)
- Can improve performance if loads are allowed to bypass previous stores (**load bypassing**) – but can do so only if the memory addresses of previous stores dispatched to the RUU are known since **all** must first check for load data dependency on previous uncommitted stores
 - Facilitated by having a common load and store queue
- Stores are not allowed to bypass previous loads, so there are **no antidependencies** between loads and stores
- Stores are committed to the D\$ in program order, so there can be **no output dependencies** between stores

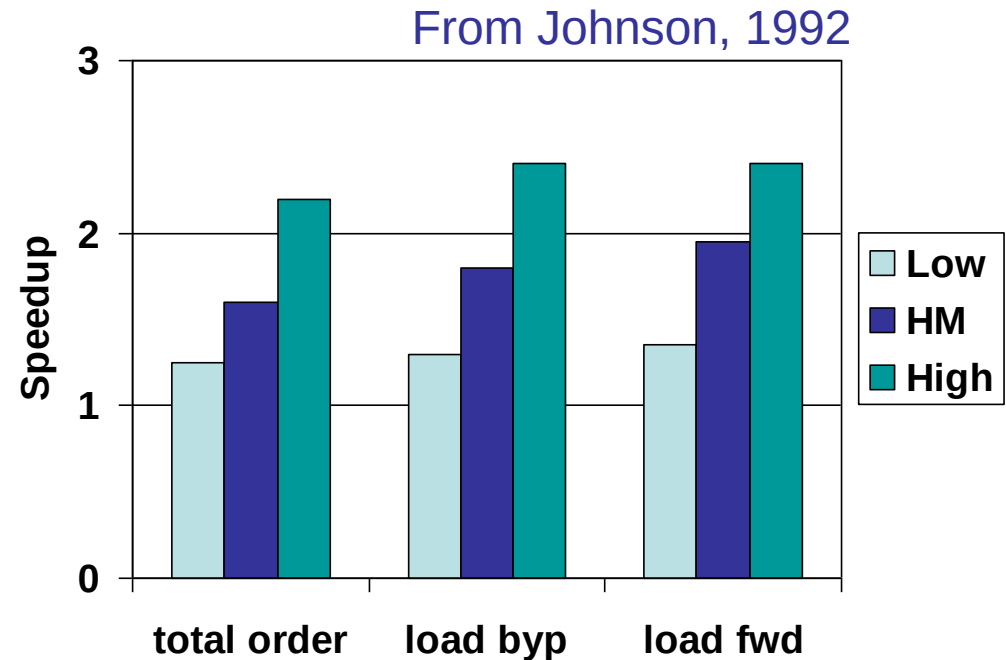
Loads from Memory

- When a load's address becomes known, the address is compared (associatively) to see if it matches an entry already in the LSQ (i.e., if there is a **pending** operation to the same memory address)
 - If the match in the LSQ is for a **load**, the current load does not need to be issued (or executed) since the matching pending load will load in the data
 - If the match in the LSQ is for a **store**, the current load does not need to be issued (or executed) since the matching pending store can directly supply the destination Content for the current load
- If there is no match, the load is issued to the LSQ and executed when the D\$ is next available
- When the RUU# of the load instr appears on the Result Bus (along with the memory data), the load completes by updating the RUU and releasing the LSQ entry (the RUU entry is released on load commit)

Load Bypassing with Forwarding

- **Load forwarding** – when a load is satisfied directly from the LSQ
 - The most recent matching LSQ data entry is supplied, since the LSQ may have more than one matching entry

- ❑ **Load bypassing** gives 19% speedup improvement with a 4-way decoder
- ❑ **Load forwarding** gives an additional 4% speedup improvement



Stores to Memory

- When a store's address (and the store data) becomes known, the address is compared (associatively) to see if it matches an entry already in the LSQ (i.e., if there is a pending operation to the same memory address)
 - If the match in the LSQ is for a **load**, the current store is **issued** to the LSQ
 - If the match in the LSQ is for a **store**, the current store is issued to the LSQ with an incremented LI
 - If there is no match, the store is dispatched to the LSQ
- Stores are held in the LSQ until the store is ready to commit (i.e., until its partner instr reaches the RUU_Head) at which time the store is executed (i.e., the data and address are sent to the D\$) and the RUU and LSQ entries are released

Summary

- ILP
 - (Super)Pipeline
 - Multi-Issue : VLIW (static) / SS(dynamic)
- Strategies in RUU to Avoid Hazard
 - Ready Bit: True data dependence
 - Latest Instance : Anti-dependence
 - In-Program-Order Commit : Output dependence
- Interaction with Memory : LSQ